Journal of Systems Architecture 179 (2026) 103911

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

A quantitative type based framework for synchronous system design☆,☆☆

Rui Chen *, Ingo Sander

School of EECS, KTH Royal Institute of Technology, Stockholm, SE-10044, Sweden

ARTICLE INFO

Keywords:
Embedded system design
Functional programming
Quantitative type theory
Proof assistant

ABSTRACT

The design process of safety-critical embedded systems is desired to be both practical and formal. These two requirements are consistently satisfied in a modern type-theory-based proof assistant/language that is increasingly accommodating general-purpose programming. A framework for embedded system design that satisfies both requirements can therefore be built upon such a proof assistant/language. In this paper, we present a framework for synchronous embedded system design that addresses both aspects (practicality and formality) by exploiting the quantitatively typed language Idris2. Specifically, we demonstrate that leveraging the expressiveness of quantitative type theory and the tagless final embedding enables an embedded domain-specific language (EDSL) named SynQ (**Syn**chronous system design with **Q**uantitative types) to be properly hosted in Idris2. This then allows Idris2 to be leveraged to facilitate synchronous system design (modelling, transformation, and implementation) and verification. It, hence, makes the presented framework a step towards an embedded system design methodology that is both practical and formal.

1. Introduction

A framework for safety-critical embedded system design requires both *practicality* and *formality* (or rigorousness according to Sifakis [1]). The former enables both *executable* models and *applicable* design steps in the design process, which can be used for virtual prototyping and design automation, respectively. The latter enables the conduct of formal specification and reasoning, ensuring the *correctness* of the designed and implemented systems. With both achieved, a correct-by-construction design methodology can be obtained.

The recent development of type-theory-based proof assistants/languages has opened new possibilities to create a unified framework in which both requirements can be satisfied. In type theory, *computations* and *formal logic*, which respectively correspond to the two needs of a system design framework, are harmoniously combined through the Curry–Howard correspondence [2]. This correspondence establishes type theory as a proper meta-theory upon which system design frameworks can be built. Furthermore, many type-theory-based proof assistants, e.g. Agda [3], Lean4 [4], and Rocq (formerly Coq) [5], are increasingly accommodating general-purpose programming. Along with this trend, general-purpose programming languages with advanced type systems, such as Idris and Idris2 [6,7], have also emerged recently. This suggests that type-theory-based proof assistants/languages, beyond their conventional use in formal specification and verification, may also be effectively employed for modelling and implementing systems, hence promoting a unified system design framework rooted in type theory that integrates reasoning and programming.

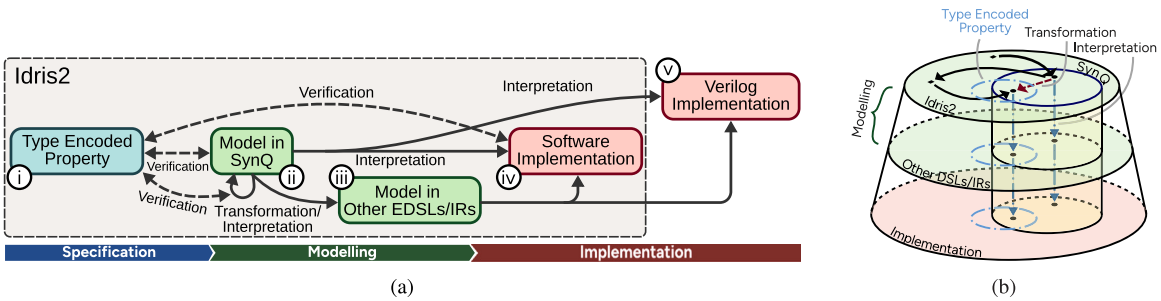
In this paper, we present a framework for synchronous system design [9,10] that aims to achieve both practicality and formality through a systematic use of Idris2 [7], a language based on quantitative type theory (QTT, a combination of linear and dependent types) [11,12]. An overview of the framework is illustrated in Fig. 1(a). In general, this framework aligns with the three-phase system design model (specification, modelling, materialisation/implementation) proposed by Sifakis [13]. The embedded domain-specific language (EDSL) SynQ (**Syn**chronous system design with **Q**uantitative types) lies at the centre of the framework and serves as the entry point ((ii) in Fig. 1(a)). SynQ is an EDSL for component-based [1] modelling of synchronous systems in Idris2. It allows systems to be modelled with QTT and the rich language and tooling features of Idris2, such as advanced type-level programming, elaborator reflection [14], and interactive editing. Moreover, it enables these models to be interpreted (implemented) and transformed (optimised) through Idris2 functions (arrows from (ii)

☆ This article is part of a Special issue entitled: 'AEiC2026' published in Journal of Systems Architecture.

☆☆ This research was partially funded by the Sweden's Innovation Agency via the NFFP7 project 2019-02743: TRANSFORM – Design transformation for correct-by-construction design methodology, and the Advanced and innovative digitalization project 2021-02484: EARLY BIRD – Seamless System Design from Concept Phase to Implementation.

* Corresponding author.
E-mail addresses: ruich@kth.se (R. Chen), ingo@kth.se (I. Sander).

https://doi.org/10.1016/j.sysarc.2026.103911
Received 13 February 2026; Received in revised form 19 June 2026; Accepted 2 July 2026
Available online 8 July 2026
1383-7621/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).



(a) (b)

Fig. 1. An overview of the proposed design framework: (a) the flowchart view of the framework, and (b) the framework in the context of the abstraction pyramid proposed in [8]. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

to (ii), (iii), (iv), and (v) in Fig. 1(a)). Thus, the practicality of the framework is achieved.

By keeping the design steps and their resulting artefacts in Idris2, the *logic aspect* of QTT is leveraged for formal specification and verification. In practice, it allows properties to be specified in types (i) and verified against models (ii, iii), implementations (iv), and design steps (solid arrows) in Idris2 by type checking. This ultimately facilitates a transparent and formally accountable design process, as advocated in [1]. In this way, the formality of the framework is achieved.

Fig. 1(b) presents another view of the framework positioned in the abstraction pyramid proposed in [8]. As shown, SynQ, as an EDSL in Idris2, serves as the entry point of the framework. On the practical side, the framework enables Idris2 to be used for modelling and transforming synchronous systems modelled in SynQ (black and red arrows), and for defining interpretations that lower the abstraction level (the blue arrows) from models to implementations in an accountable way. On the formal side, the framework allows type encoded properties to be specified and verified against SynQ models (the blue circle) and transformations. Such a property can also be propagated to other abstraction levels through formally accountable interpretations, which, thus, preserves the correctness of the design process across abstraction levels.

Taken together, the practical and formal aspects motivate a key pillar of our framework: the embedding of SynQ in Idris2 should be *sufficiently shallow*, which intuitively means that a synchronous system is represented by a function in Idris2, rather than as a data-encoded abstract syntax tree as in a deep embedding (cf. [15] for detailed discussion about deep and shallow embeddings). Thus, models, implementations, and the design steps between them remain first-class Idris2 terms, which can be executed and verified within the same type theory. To achieve such a proper embedding of SynQ at Idris2's term level and to leverage Idris2 in the proposed SynQ-based system design framework, *three specific challenges* arise. These are to be addressed by QTT and the tagless final embedding in this paper.

The *first challenge* (**C1**) is to guarantee that terms in SynQ are of a *closed* subset of Idris2 terms. By closed, we mean that SynQ terms are built upon only other SynQ terms. If achieved, interpretations and transformations within the design framework can focus only on SynQ terms, rather than on all constructors in Idris2. This, however, is not the case when system models are represented as functions in Idris2 because, from the Idris2 point of view, a function can use arbitrary constructs, such as pattern matching on types and the `with` clause, as long as it is well-typed.

The *second challenge* (**C2**) is to allow SynQ terms, as Idris2 functions, to be interpreted to exhibit separate functional behaviour that differs from the behaviour entailed in Idris2's implementation. In this way, interpretations of SynQ terms are prevented from inheriting implementation details of Idris2. It allows different interpretations of a SynQ term to have consistent, desired behaviour. Otherwise, the evaluation of functions must obey the call-by-value [16, p. 57] evaluation strategy implemented in Idris2, which is not always the desired behaviour for interpretations of SynQ terms. For instance, interpreting a model to digital circuits would require call-by-need [16, p. 57] evaluation with observable sharing [17] to better handle potential loop structures.

The *last challenge* (**C3**) is to implement design steps that manipulate SynQ terms, such as the interpretations and transformations illustrated in Fig. 1, as Idris2 terms as well. This enables the formal verification of design steps in Idris2 and, hence, a formally accountable design process that is defined and implemented in Idris2. Furthermore, by keeping both design steps and terms manipulated by these steps as Idris2 terms, transformations can be easily composed and applied just as functions on SynQ terms. This suggests the *semantic coherence* that the design process defined by these design steps has, which, according to Sifakis, is a key enabler for *system design automation* [1]. However, addressing this challenge is a non-trivial task because manipulating SynQ terms, which are functions in Idris2, requires inspecting and manipulating the structure of those functions. This often involves meta-programming techniques that are hard to be verified in Idris2's type system.

We summarise the contributions as follows:

- this paper presents SynQ, an EDSL for modelling synchronous systems with a focus on how QTT, combined with the tagless final approach, is leveraged so that challenges (**C1**/**C2**) above are addressed;
- this paper showcases that, with SynQ as the enabler, QTT can be leveraged to establish a framework in which both design (modelling and implementation) and formal specification/verification can be conducted; and
- this paper showcases, with examples, that design steps for SynQ terms, such as refinements and transformations, can be defined and implemented as functions in Idris2, with their correctness facilitated by leveraging QTT (challenge **C3**).

By combining these contributions, the paper showcases that a practical and formal framework for synchronous system design can be achieved through the systematic use of a modern type-theory-based language, i.e., Idris2. The authors, thus, believe that this paper forms a step towards a formally accountable embedded system design methodology [1].

The rest of the paper is organised as follows. Section 2 discusses related work to clarify the positioning of this research. Section 3 introduces quantitative type theory and synchronous systems. Section 4 provides an illustrative example to overview SynQ and the system design framework. Section 5 details the embedding of SynQ and addresses the first two challenges discussed above. Section 6 presents transformations of SynQ terms to resolve challenge **C3**. Next, Section 7 showcases three examples illustrating how SynQ can be used for system design, verification, and how the transformations discussed may be applied. Finally, Section 8 provides further discussion about the work, and Section 9 concludes the paper with directions for future work.

It is worth mentioning here that a previous paper covering the first half of the topic of this work was published in SETTA-2025 [18]. Compared to that paper, in this work, the embedding of SynQ (Section 5) is fully rewritten, presented with more intuitions, and adapted

to other contents. Other related sections (Sections 3, 4 and 7.2) are also revised and elaborated. Based on the revised contents, this paper further presents and demonstrates transformations of SynQ terms, their verification, and application (Sections 6 and 7.3). These revised and newly presented contents, together, form a more comprehensive presentation on how a *framework* for synchronous system design can be achieved by leveraging Idris2 and QTT.

2. Related work

Theorem proving has been demonstrated to be a realistic approach for building high-assurance software through many projects, such as seL4 [19], CompCert [20], and Vélus [21]. In these projects, a microkernel (seL4) and compilers for C (CompCert) and Lustre (Vélus) are built and verified. The latter two involve programs written directly in Rocq (formerly Coq) [5], using the proof assistant similarly to how Idris2 is used in this paper. While those projects focus on the verified final products, such as compiler, kernel, etc., this paper demonstrates how a framework for the design of safety-critical embedded systems can be defined in an expressive type theory.

Besides proof assistants, functional programming languages have been employed for modelling both synchronous systems and digital circuits. Lucid-Synchrone [22] is a synchronous language built upon OCaml that extends Lustre with higher-level mechanisms provided by OCaml, such as types and higher-order functions. Besides the original work, there is also a shallow embedding of Lucid-Synchrone in Rocq [23] relying on the *clocks as types* paradigm [24]. ForSyDe [25,26] is a framework that allows models of computation [27], including the synchronous model of computation, to be modelled in Haskell. In [25], transformations of ForSyDe terms are also discussed. These design transformations, including strategies about how they are applied, are later implemented in [28–30]. Their implementation relies on an intermediate representation named ForSyDe IO [31]. Lava [32–34] is a family of EDSLs in Haskell for digital circuit design. The underlying idea of Lava that observing (a subset of) Haskell functions as digital circuits is also shared by SynQ. Transformations of Lava programs are investigated in [35], which are implemented based on the Core language (the intermediate representation used by the Haskell compiler) [36]. In [37], the κ-calculus is employed to build a DSL for digital circuit design in Haskell. It allows recursive structures in circuits to be specified by higher-order functions in Haskell, while ensuring that the results remain within the κ-calculus, and hence are all well-formed digital circuits.

Compared to the above-mentioned works, SynQ uniquely allows its term transformations to be implemented as functions in its host language, Idris2. Conversely, the above-mentioned works either do not consider transformations of their terms, e.g., Lucid-Synchrone [22,24] and the research presented in [37], or require transformations to be implemented externally on an intermediate representation, such as ForSyDe (based on ForSyDe IO [31]) and Lava (based on the Haskell Core language [35]). Implementing transformations as Idris2 functions benefits the design process of synchronous systems, as it allows transformations to be applied explicitly in a SynQ model (presented in Section 7.3). This eventually facilitates the *transparency* and *accountability* of a design process, which are required for safety-critical embedded system design [1].

Meanwhile, the formality of Idris2 enables formal specification and verification of both SynQ terms and their transformations within the same language. On this aspect, the shallowly embedded Lucid-Synchrone [23], which employs Rocq to give Lucid-Synchrone a formal denotational semantics, is similar to SynQ. However, [23] does not consider transformations of Lucid-Synchrone terms, and hence does not leverage the same language for both design and transformation. Most prior works, by contrast, are limited by their meta-language and embedding, and thus have to conduct formal verification externally, e.g., model checking via SMV models extracted from ForSyDe models [38] and symbolic interpretation of Lava terms [32]. Translating designs to different languages for verification risks severing the connection between a design and its proof. Conversely, verification in the same language as the design, as enabled by SynQ and Idris2, can maintain the connection and enhance the accountability of the design process. Thus, unlike prior work that employs functional programming solely for the DSL, this paper presents an integrated framework where the DSL (SynQ) and its design steps (transformations, interpretations, and verification) are jointly embedded and verified as first-class terms in Idris2.

3. Preliminaries

3.1. Quantitative types

Quantitative type theory (QTT) [11,12] is a combination of dependent types and linear types [39]. We now briefly introduce QTT. The interested reader is referred to [11,12,39,40] for further details.

Dependent types allow types to be constructed from terms and vice versa, enabling extra information to be specified in types. For instance, we can define type `Vect n a` for all vectors of length `n` (which is a term of type `Nat`/natural numbers) whose elements are of type `a`. The expressiveness of dependent types allows the Curry–Howard correspondence to be leveraged to reason about programs' behaviour. Intuitively, the Curry–Howard correspondence states that type-checking a term t of type T corresponds to proving a proposition $\mathcal{P}_T$ in a corresponding proof calculus, with t itself serving as the witness of $\mathcal{P}_T$. With dependent types, propositions over terms (programs) can be encoded in types and formally proved. For instance, consider the following proposition:

Proposition 1. *There exists a natural number n such that $n + 1 = 2$, i.e., $\exists n : \mathbb{N}.\, n + 1 \equiv 2$.*

It is encoded in dependent types as:

```
prop1: (n: Nat ** n + 1 = 2)
```

Here, the dependent pair type `x ** P` encodes the existence quantifier $\exists x.\, P$, i.e., there exists x (instantiated by n in the proposition) such that P ($n + 1 \equiv 2$ in the proposition) holds. The propositional equality is encoded by the equality type of the form `x = y` whose term, `Refl`, is well-typed iff x and y are equal. A term of type `prop1`, which proves the proposition, is hence constructed from a concrete natural number n and a proof that $n + 1 = 2$, i.e.:

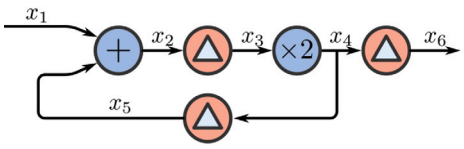
```
prop1 = (1 ** Refl)
```

In this way, dependent types allow properties to be specified in types and verified by type checking.

The *linear type system* restricts terms to be used *exactly once (linearity)*. Combining dependent and linear types is non-trivial because the former allows terms to appear in types. QTT addresses this issue by associating terms with *multiplicities* on their type bindings to specify the usage of terms. These multiplicities form a semiring, using addition to sum multiple uses of a term and multiplication for nested uses. Associating terms with the multiplicity corresponding to the absorbing element of multiplication (0) removes restrictions on term usage, thus permitting the combination of linear and dependent types. Specifically, Idris2 implements QTT with the $\{0, 1, \omega\}$ semiring, with regular terms bound with multiplicity ω, terms that can be used only in types bound with multiplicity 0, and linear terms bound with multiplicity 1.

Unlike conventional type systems, types in QTT can restrict *how a well-typed function can be implemented*, which is desirable for system modelling. For example, consider a function `length` that calculates the length of vectors declared as:

```
length: {0 n: Nat} -> (1 vec: Vect n a) -> Nat
```



	t_1	t_2	t_3	t_4	t_5	t_6	...
x_1	1	2	3	4	5	6	...
x_2	1	2	5	8	15	22	...
x_3	0	1	2	5	8	15	...
x_4	0	2	4	10	16	30	...
x_5	0	0	2	4	10	16	...
x_6	0	0	2	4	10	16	...

Fig. 2. An exemplified synchronous system (left) illustrated as a network consists of combinational operators (blue) and unit delays (red); and streams' value at each clock event by assuming all unit delays have initial value 0. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Here, `0` and `1` are the multiplicities associated with the type bindings of variables `n` and `vec`, respectively, and `{_ n: Nat}` specifies that `n` is an *implicit parameter* of `length` which may be resolved by the type checker when `length` is used. Since a vector type is parametrised by its length, without considering restrictions introduced by multiplicities, a straightforward implementation of `length` can be:

```
length {n} vec = n
```

This, however, will be *rejected* in QTT because `n`, which is bound with multiplicity 0, *is used* as the return value of the function; meanwhile, `vec`, which is bound with multiplicity 1, *is not used* in the function's implementation. The type signature hence restricts `length` to be implemented by pattern-matching `vec` to the empty and non-empty cases, i.e.:

```
length {n=0} [] = 0
length {n=(S k)} (v::vec') = 1 + (length vec')
```

Note that pattern-matching allows linear terms to be eliminated. Hence, by pattern-matching on `vec`, the linearity constraint on `vec` is discharged, and its content can be used freely. This feature allows the *of course modality*, whereby terms can be used unrestrictedly, to be defined as follows:

```
data (!*) : Type -> Type where
  MkBang: (unrestricted: a) -> !* a
```

With this type, a linear term bound as `(1 x: !* a)` can have its inner content of type `a` used unrestrictedly via pattern matching.

3.2. Synchronous systems

Synchronous systems are reactive systems obeying the *perfect synchrony hypothesis* that "reactions are instantaneous so that activations and productions of output are synchronous, as if programs were executed on an infinitely fast machine" [41]. In other words, a synchronous model abstracts away (physical) time consumed by computations, while preserving the order between a system's input and output events. This implies a potentially aperiodic, abstract *global clock* synchronising the entire system.

A synchronous system's behaviour can be specified by the dataflow model as a set of equations over *synchronous isomorphic* (cf. Section 3 of [42]) streams (sequences of events, dataflow, cf. the Kahn process network [43] and Lustre [9]) that are mutually recursive. For instance, the following set of equations defines a synchronous system:

$$x_2 = x_1 + x_5 \qquad x_4 = x_3 \times 2 \qquad x_6 = D\, x_4$$
$$x_3 = D\, x_2 \qquad x_5 = D\, x_4$$

which can be directly interpreted as a netlist and visualised as a network illustrated in Fig. 2. In this set of equations, x_{1-6} are streams such that $x_1[k]$ refers to the value of x_1 at the kth clock event, and the constant 2 represents a constant stream of value 2. Operators $(+)$ and $(\times)$ indicate *point-wise* (combinational) operations over streams and D is the primitive of unit delay such that $(D\, x)[k] = x[k-1]$ with a pre-defined $(D\, x)[0]$.

4. An illustrative example

We use an example to provide an overview of SynQ and the framework it enables. This example demonstrates a system that outputs "1" upon detecting a *rising pattern* in its input. Following Section 3.2, both the input and output are streams obeying synchronous isomorphism. The expected behaviour of the system, i.e., rising pattern detection, can then be defined as follows (for all $k > 0$)

$$\texttt{xout}[k] = \begin{cases} 1 & \text{if } \texttt{xin}[k-1] < \texttt{xin}[k] \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where `xin` and `xout` are the input and output stream, respectively. The design process of this system, along with the artefacts/results involved in each step, is presented in Fig. 3.

Fig. 3b illustrates the system model in SynQ (`isRising`) and its visualisation as a network. This model intuitively corresponds to the following equation

$$\texttt{xout} = (D\ \texttt{xin}) < \texttt{xin}$$

in which $(<)$ denotes the pairwise assertion of the *less-than* relation. Compared to the equation-based model, the SynQ model in Fig. 3b is elaborated with types and focuses more on the structural description of the system modelled as a network. Specifically, in the SynQ model, the type of `isRising` (line 1–3 of Fig. 3b) explicitly specifies that:

- the set of components (basic building blocks) used to construct `isRising`, specified by the parameters `Seq comb seq`, `Primitive comb`, and `Reg UInt8 comb seq` (line 1 and 2, cf. Section 5.2 and Figs. 8 and 9);
- `isRising` consists of exactly one delay element (shown by multiplicity 1 in line 2); and
- the resulting system model has input, output and internal state of type `UInt8`, `BitVec 1`, and `!* UInt8`, respectively (line 3).

From line 4 to line 8, the model is given. In line 4, the term `(MkReg get set)` unpacks the register (`reg`) declared in the type signature, exposing its `get` (output) and `set` (input) primitives for system construction. Then, at line 5, the `abst` expression introduces the variable `xin` as the system's input. From line 6, a structural description (model) of the system is given. It can be directly related to the network illustrated on the right of Fig. 3b by interpreting a term of the form `x <- f` to mean that the output of `f` is named `x`.

Fig. 3a illustrates a type-encoded property that `isRising` should have if it correctly implements the behaviour specified by Eq. (1). Following the discussion in Section 3.1, this type corresponds to the following proposition:

$$\forall\, \texttt{st}, \texttt{xs}, \texttt{x}, \texttt{y}.\ (\texttt{x} \overline{<} \texttt{y}) \equiv 1$$
$$\implies \exists\, \texttt{ys}.\ [\![\texttt{isRising}]\!]_{\texttt{st}}\ (\texttt{xs} +\!\!+ \texttt{x} +\!\!+ \texttt{y})$$
$$\equiv \texttt{ys} +\!\!+ 1$$

In which $[\![isRising]\!]_{st}$ (`isRisingMealy` in Fig. 3a) denotes the functional behaviour of `isRising` initialised with state `st` (cf. Section 5.1). It specifies that if $\texttt{x} < \texttt{y}$, then the output sequence of

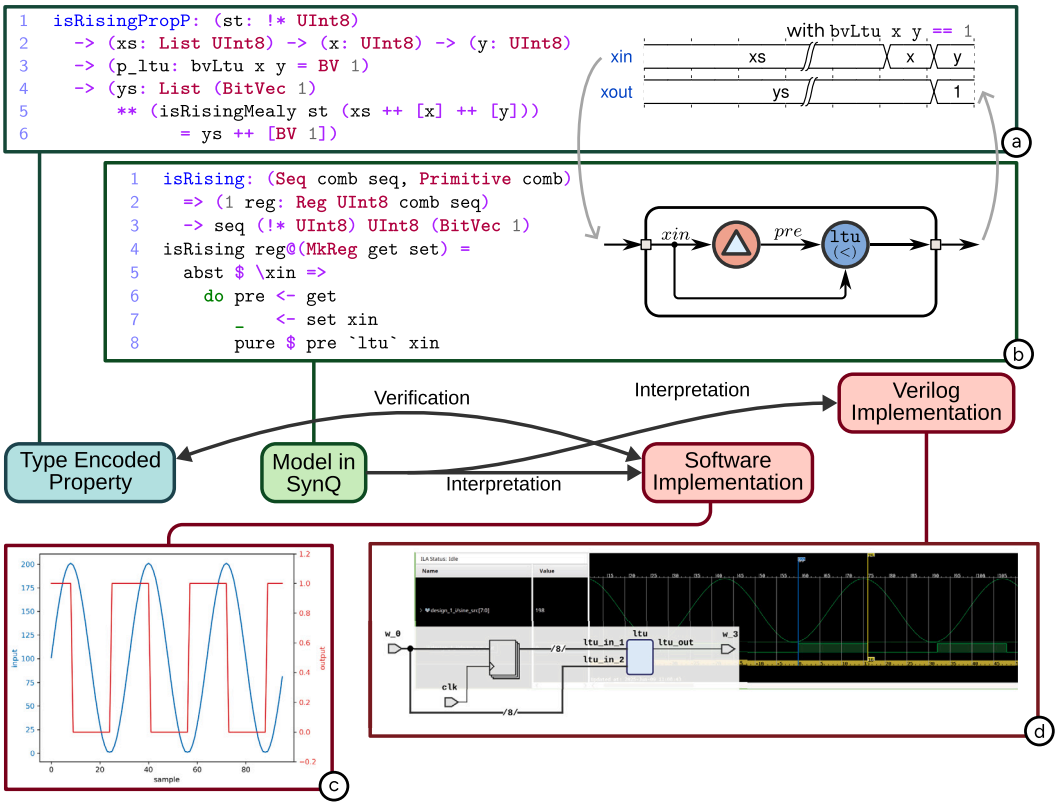


Fig. 3. An illustration of the design of a rising pattern detecting system: (a) the specification of the functional behaviour of the system's software implementation in QTT; (b) the SynQ model of the system; (c) and (d) the illustration of the input–output behaviour of the software and hardware implemented system. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

the system produced by consuming an arbitrary input sequence `xs` followed by `x`, then `y`, always ends with `1`. The type-encoded property is verified/proved in Idris2 by pattern matching on the prefix of the input (`xs`), i.e.:

```
isRisingPropP (MkBang st) [] x y pltu =
  rewrite p_ltu in ([bvLtu st x] ** Refl)
isRisingPropP (MkBang st) (st' :: xs) x y pltu =
  let (ys ** prf) = isRisingPropP (MkBang st') xs x y pltu
  in rewrite prf in (bvLtu st st' :: ys ** Refl)
```

The first two lines verify the case where the input is just `x` followed by `y` (the base case). Meanwhile, the rest inductively proves that the property holds for arbitrary non-empty `xs`.

Two interpretations are made for the model, which gives us the software and hardware implementation, respectively. The software implementation is an Idris2 program that invokes C-implemented components (e.g., `bvLtu`) to efficiently leverage platform primitives. The result program can be verified or tested against the type-encoded property. Fig. 3c illustrates the input/output behaviour, in red and blue, respectively, of this implementation. The hardware implementation of the system is a synthesizable Verilog HDL file, which can be synthesised and deployed as a digital circuit. The visualisation of the generated HDL file and the input/output waveform of the FPGA-deployed system captured by a logic analyser is presented in Fig. 3d.

5. SynQ and its embedding

We now present SynQ, the EDSL at the centre of the proposed framework for synchronous system design in Idris2. SynQ uses a small set of constructors to capture an expressive subset of synchronous systems, where implementations follow from implementing the constructors and transformations follow from equivalences between their combinations This idea is aligned with component-based design [1]. In this section, we first introduce the small set of constructors employed by SynQ. The introduction is based on a purely shallow embedding to convey the core idea of modelling synchronous systems with these constructors. After that, we introduce the tagless final embedding of SynQ, and show that it realises the idea with challenges **C1** and **C2** addressed.

5.1. Synchronous systems in SynQ

As illustrated in Section 3.2, a synchronous system can be represented as a network, namely a process network [43], whose edges are streams obeying the *synchronous isomorphism* [42, sec. 3], and nodes (processes) are operations over those streams. Specifically in SynQ, synchronous systems are modelled as process networks where each process is a *Mealy machine*. These Mealy machines can be intuitively specified by functions of the following type in Idris2 (the type is named *Mealy* for brevity)

```
Mealy : Type -> Type -> Type -> Type
Mealy s a b = a -> (1 st: s) -> LC s b
```

with LC (linear context) defined as the following, which is, in essence, a pair of a state `st: s` bound with multiplicity 1 and a value `val: a` that can be used unrestrictedly:

```
data LC : Type -> Type -> Type where
  (#) : (1 st: s) -> (val: a) -> LC s a
```

Here, `(#)` is an infix operator, i.e., terms of the type `LC s a` are of the form `st # val`. With this type, the specification of a Mealy machine of type `Mealy s a b` is a function that takes two parameters: an input of type `a` and a linear state of type `s`, and produces an output of type `b` together with an updated state of type `s` wrapped in the type `LC`. Unrestricted inputs and outputs allow free interconnection between Mealy machines, while linear states ensure proper control over firing

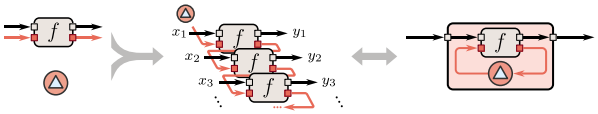
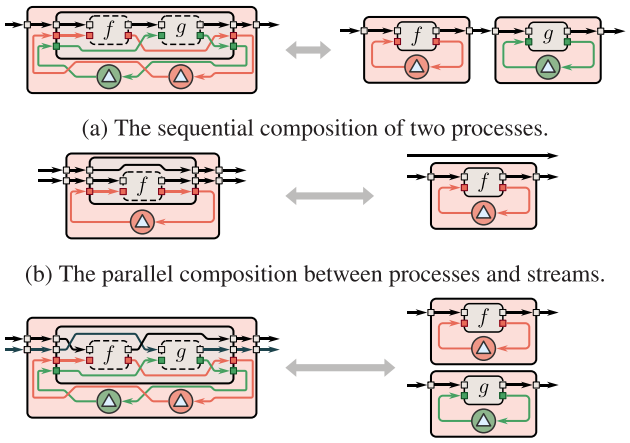


Fig. 4. The visualisation of the function `run`.



(a) The sequential composition of two processes.

(b) The parallel composition between processes and streams.

(c) The parallel composition of two processes.

Fig. 5. Compositions of processes at the single-step behaviour (left) and stream (right) level.

and composition. Note that a function of type `Mealy s a b` specifies only the single-step behaviour (per clock event) of a process.

Compositions of process networks are generally defined at the stream/sequence level. For instance, in [43], the semantics of Kahn process networks (KPNs) are defined based on the prefix order of sequences, and in [27], the behaviour of process networks is defined by the tagged event model with partially ordered tags, which can be regarded as sequences at the input/output of processes. For synchronous processes specified by `Mealy s a b`, however, composing single-step behaviour suffices to define sequential and parallel network compositions. These compositions, together with a few basic processes, allow a subset of synchronous process networks that are sufficiently expressive to be specified. This forms the basis of SynQ.

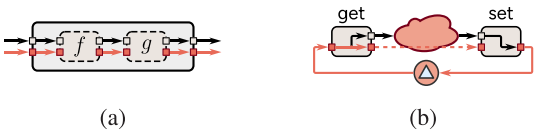
To make this clear, we first consider the function `run`, which *lifts* a function of type `Mealy s a b` to the stream level, i.e., applies it to a sequence of inputs given an initial state, yielding the sequence of outputs and the final state:

```
run: (f: Mealy s a b) -> (1 st: s)
  -> Vect n a -> LC s (Vect n b)
```

This function is defined as the following, which can be intuitively visualised as in Fig. 4:

```
run f st [] = st # []
run f st (x :: xs) = let (st' # y) = f x st
                         (st'' # ys) = run f st' xs
                     in st'' # (y :: ys)
```

Here, input and output streams are encoded as vectors of length n, differing slightly from formalisms that use infinite sequences [44]. This is sufficient and preferred here for two reasons: (1) arbitrary-length behaviour is still captured since n is arbitrary; and (2) fixing input and output lengths to n makes their one-to-one correspondence explicit.



(a) (b)

Fig. 6. The sequential composition of components sharing the same state (a), and the loop formed by sequentially composing `get` and `set` sharing the same state (b).

We can now introduce compositions defined on specifications of processes' single-step behaviours and make their relation to compositions of lifted processes explicit, which are drawn by the following propositions[1] and illustrated in Fig. 5.

Proposition 2. *Lifting two processes' single-step behaviours that are sequentially composed (left of Fig. 5(a)) is equivalent to the sequential composition of the lifted processes (right of Fig. 5(a)).*

Proposition 3. *Adding an extra pass-through input/output to a process's single-step behaviour (left of Fig. 5(b)) is equivalent, up to the synchronous isomorphism, to the parallel composition of the process with a stream (or an identity process, right of Fig. 5(b)).*

Finally, as a corollary of Propositions 2 and 3:

Proposition 4. *Lifting two processes' single-step behaviours that are composed in parallel (left of Fig. 5(c)) is equivalent, up to the synchronous isomorphism, to the parallel composition of the two processes (right of Fig. 5(c)).*

To ensure expressiveness, *feedback loops* that close a process's input/output should also be supported. Such a feedback loop, however, cannot be defined in the same form as sequential and parallel compositions, as it introduces a circular dependency between a process's input and output. Thus, we introduce such feedback loops by a two-part component, namely `Reg`, defined as the following:

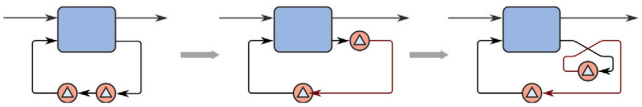
```
record Reg a where
  constructor MkReg
  1 get : Mealy (!* a) () a
  1 set : Mealy (!* a) a ()
```

The (sub-)components `get` and `set` retrieve and update the current state, respectively. Because of the multiplicity 1 assigned to these (sub-)components, the type system enforces that `get` and `set` sharing the same state are used together. A feedback loop is then introduced when a process referring to `get` and `set` is lifted to the stream level by `run`. In other words, `get` and `set` respectively correspond to the output and input of a delay element and allow them to be accessed in a process' single-step behaviour specification. Meanwhile, the linear restriction on `get` and `set` guarantees that a delay element can be properly treated as a resource. Finally, the composition shown in Fig. 6(a) is introduced to compose components sharing the same state, such as `get` and `set`. Note that, in the general case, the state sharing in the forward path is conceptual and is used to pair the components. It is the backward path introduced by the function `run` that matters. For instance, one may consider a sequential composition of the two parts constituting in a `Reg` component introduces a loop at the sequence level in the way illustrated in Fig. 6(b).

[1] The interested reader may find the proof in Idris2 in https://gist.github.com/ruich95/4ef72ae406dfbe2f7ed97cd9136f91b9.



(a) The transformation of a forward delay.



(b) The transformation of a feedback loop that consists of two delays.

Fig. 7. An illustration of transforming general synchronous models into SynQ models.

The subset of synchronous process networks identified. With these compositions and components, the subset of synchronous process networks that can be specified includes all the networks in which delay elements are only used in feedback loops, and there is exactly one delay element in each feedback loop. This is because `run` is the only place where delays and feedback are realised; `Reg a` merely exposes the interfaces that `run` connects. Even though this subset is restricted, other synchronous models can be transformed into equivalent models within it. For instance, forward delays, such as the one in Fig. 3b, can be transformed and introduced as a component as illustrated in Fig. 7(a). Meanwhile, a feedback loop with multiple delays can also be transformed into the desired form. Fig. 7(b) illustrates the case where a feedback loop with two delays is transformed into the desired form. This subset retains an expressiveness that is similar to Lustre, in which only zero-delay loops are forbidden [9].

5.2. The embedding of SynQ

We have now illustrated the small set of constructors for modelling synchronous systems, including the single-step behaviour specification of Mealy machines, four compositions (Figs. 5(a) to 5(c) and 6(a)), a primitive component `Reg a`, and the lift function `run`. However, a purely shallow embedding, despite allowing processes to be specified as functions of type `Mealy s a b` and QTT to restrict the implementation space and verify properties, does not support building a design framework on top of it.

To highlight the challenges posed by the shallow embedding, we may consider a shallow version of `isRising` presented in Fig. 3b, which is defined as the following:

```
isRising': (1 reg: Reg UInt8)
        -> Mealy (!* UInt8) UInt8 (BitVec 1)
isRising' (MkReg set get) x (MkBang prev) =
  let ((MkBang _) # _) = get () (MkBang 0)
      ((MkBang _) # _) = set 0  (MkBang 0)
  in (MkBang x) # (x > prev)
```

`isRising'` is well-typed with the types introduced in the preceding subsection, and its type signature carries the same intuition as that of `isRising`. However, it is not constructed from the previously discussed constructors, such as the compositions and the primitive component `Reg a`. Thus, `isRising'` cannot be implemented on a target platform by implementing constructors and interpreting it, nor transformed by exploiting equivalences between constructor combinations. This example highlights the difficulty of building a design framework over a non-closed set of constructors, which reflects challenge **C1**.

The shallow embedding also raises challenge **C2**. This is because a shallow embedding denotes the embedded language using the meta-language's semantics, fixing the embedded language's semantics accordingly. In other words, with shallow embedding, a constructor is implemented directly as a function specifying its desired behaviour in the meta-language. Thus, there is no space for providing alternative interpretations of a specified constructor within the same language. These difficulties motivate the tagless final embedding of SynQ. In the rest of this section, we first introduce the tagless final embedding of SynQ, together with how it addresses challenge **C1**, and then discuss how challenge **C2** is addressed.

```
interface Comb (0 comb: Type -> Type -> Type) where
  lam: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
    -> (comb () a -> comb () b) -> comb a b
  app: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
    -> comb a b -> comb () a -> comb () b
  unit: comb () ()
  prod: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
    -> comb () a -> comb () b -> comb () (a, b)
  proj1: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
    -> comb () (a, b) -> comb () a
  ...
```

(a)

```
interface Primitive (0 comb: Type -> Type -> Type) where
  const: {n: Nat} -> BitVec n -> comb () (BitVec n)
  ltu: {n: Nat} -> comb () (BitVec n) -> comb () (BitVec n)
    -> comb () (BitVec 1)
  ...
```

(b)

Fig. 8. A fragment of the symantics of the combinational layer of SynQ: (a) the interface that defines the symantics of combinational glue components; (b) the interface that defines the symantics of combinational atomic components.

In the tagless final approach, a language is introduced by declaring interface(s), better known as *type classes* in Haskell [45–47], parametrised by type constructors that specify terms in the language. Such interface(s) are recognised as the **symantics** (**sy**ntax + se**mantics**) of the language because it constrains both aspects of the language. Specifically, SynQ is embedded with four interfaces, namely `Comb`, `Primitive`, `Seq`, and `Reg`, grouped into two layers: the combinational layer (`Comb` and `Primitive`) and the sequential layer (`Seq` and `Reg`). Following the convention of component-based design, `Comb` and `Primitive` respectively provide the glue and atomic components for the combinational layer, while `Seq` and `Reg` correspondingly provide the glue and atomic components for the sequential layer.

5.2.1. Combinational layer

A fragment of the symantics of this layer is illustrated in Fig. 8. With this symantics, side-effect-free systems (components) are modelled in this layer, which intuitively defines single-step computational behaviour of processes. To guarantee well-defined behaviour within each clock-event, this layer employs a subset of *simply typed lambda calculus (STLC) with product* as glue components (`Comb`) and *pure-functional constants* as atomic components (`Primitive`). In this way, the strong normalisation property of STLC (cf. [48, Part I], also for the Church–Rosser theorem and β/η-equivalence), together with the Church–Rosser theorem, can be leveraged for specifying deterministic computations with inherent parallelism. Meanwhile, well-established equivalence relations defined on lambda terms, such as β/η-equivalence, allows equivalence relations between different systems to be easily defined and explored, which, in turn, can be used for obtaining a sound design space of a system.

A component in the combinational layer, namely `foo`, with input and output of type `a` and `b` respectively, is normally given with the following type:

```
foo: {0 comb: Type -> Type -> Type}
  -> (Comb comb, Primitive comb)
  => comb a b
```

```
interface Comb comb
  => Seq (0 comb: Type -> Type -> Type)
         (0 seq: Type -> Type -> Type -> Type) | seq where
  abst : {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
      -> {auto sIsState: St s}
      -> (1 _: comb () a -> seq s () b) -> seq s a b
  (<<<): ... -> (1 _: seq s2 b c) -> (1 _: seq s1 a b)
      -> seq (LPair s1 s2) a c
  pure : ... -> comb a b -> seq s a b
  (=<<): ... -> (1 _: seq s b c) -> (1 _: seq s a b)
    -> seq s a c
```

(a)

```
interface Comb comb
  => Reg (0 a: Type) (0 comb: Type -> Type -> Type)
         (0 seq: Type -> Type -> Type -> Type) | seq where
  constructor MkReg
  get: {auto aIsSig: Sig a} -> {auto sIsState: St s}
    -> {auto similar: SameShape a s} -> seq s () a
  1 set: ... -> comb () a -> seq s () ()
```

(b)

Fig. 9. The symantics of the sequential layer of SynQ: (a) the interface that defines the symantics of sequential glue components; (b) the interface that defines the symantics of the sequential atomic component Reg. Note that some implicit parameters are omitted for brevity.

Here, `comb a b` parameterises the component type with `comb`, a type constructor constrained by `Comb` and `Primitive`. That is, `foo` is an abstract component interpretable by any `Comb` and `Primitive` instance, such as `(->)` (the function type constructor), with each interpretation representing an implementation or refinement of the component. Here, being an instance of `Comb` and `Primitive` means that the concrete type constructor provides implementations of all methods declared in these interfaces, with the occurrence of the variable `comb` substituted by the concrete type constructor itself. For instance, instantiating `Comb` with the function type constructor `(->)` requires functions such as the following to be implemented (where the red arrows are introduced by the substitution of `comb` by `(->)`):

```
lam: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
  -> ((() -> a) -> (() -> b)) -> (a -> b)
app: {auto aIsSig: Sig a} -> {auto bIsSig: Sig b}
  -> (a -> b) -> (() -> a) -> (() -> b)
...
```

Since `comb` can be any `Comb` and `Primitive` instance, all methods in these interfaces are available for implementing `foo`. Meanwhile, since the first (implicit) parameter of `foo`, i.e., the binding of `comb`, is bound with multiplicity 0, the value of `comb` may not be used in the implementation of `foo`. This makes the availability of the methods declared in `Comb` and `Primitive` the **only** information about `comb` that can be utilised in the implementation of `foo`. It guarantees that the implementation of `foo` is constructed solely by using the methods declared in `Comb` and `Primitive`, and, thus, addresses challenge **C1**.

5.2.2. Sequential layer

Symantics of the sequential layer are illustrated in Fig. 9. This layer realises the ideas from Section 5.1 using tagless final embedding. Specifically, the aforementioned `Mealy` type constructor is abstracted to a variable `seq` constrained by the interfaces `Seq` and `Reg` in the embedding. Compositions illustrated/discussed in Fig. 5(a) (Proposition 2), Fig. 5(b) (Proposition 3), and Fig. 6(a) are realised by the methods/glue components `(<<<)`, `pure` and `(=<<)` declared in the interface `Seq`, respectively.[2] And the primitive component `Reg a` is realised by the interface `Reg` (atomic component in the sequential layer). This consequently allows the same subset of synchronous process networks as discussed in Section 5.1 to be specified in the sequential layer of SynQ.

A component in this layer, such as the `isRising'` discussed at the beginning of this subsection, is typed as the follows (the same as the one of `isRising` presented in Fig. 3b, except that implicit parameters are written explicitly), which can only be implemented with constructors defined in SynQ:

```
isRising': {0 seq: Type -> Type -> Type -> Type}
        -> {0 comb: Type -> Type -> Type}
        -> (Seq comb seq, Primitive comb)
        => (1 reg: Reg UInt8 comb seq)
        -> seq (!* UInt8) UInt8 Bool
```

5.2.3. Addressing challenge C2

Challenge **C2** has two parts: decoupling SynQ models from Idris2's semantics, and ensuring behavioural consistency across implementations. SynQ addresses the first part via tagless final embedding, together with two glue components, `lam` and `abst`, defined in Figs. 8(a) and 9(a) respectively. The second part is addressed by exploiting the expressiveness of QTT to introduce behavioural constraints at the symantics level, which will be inherited by all implementations.

Specifically, recall from previous discussions that systems are modelled by abstract type constructors, such as `comb` and `seq`, constrained by interfaces. Interpretations are required to map these abstract type constructors to concrete type constructors. These interpretations and the choice of concrete type constructors introduce a degree of freedom in the semantics of SynQ models. For instance, `comb a b` can be mapped not only to `a -> b`, but also to `a -> State ctx b`, where `State ctx` is the state monad [49] capturing the execution context. This allows SynQ models to have behaviour that is different from Idris2. Meanwhile, the glue components `lam` and `abst`, as suggested by their type signatures, allow Idris2 functions to be transformed into components in the combinational and sequential layers. This is formally recognised as the higher-order abstract syntax (HOAS) [50].

Behavioural consistency across multiple implementations is achieved by those implicit parameters, such as `{auto aIsSig: Sig a}` and `{auto sIsState: St s}`, and multiplicity 1 employed in the symantics of SynQ (Figs. 8 and 9). These implicit parameters and multiplicity 1 restrict both the construction of SynQ models and the implementation of the symantics. Since interpreting SynQ terms is conducted by implementing the symantics, these restrictions are inherited by all implementations. For instance, the implicit parameters `{auto aIsSig: Sig a}` restrict the type variable `a` to range over only few value types, such as bit-vectors and their products, which consequently restricts the combinational layer (and all its implementations) to be a subset of STLC excluding higher-order functions (of a type like `comb (comb a b) c`). Multiplicity 1 places an even stronger restriction on components in the sequential layer. For instance, when glues `(<<<)` and `(=<<)` are implemented for `Mealy`, the linearity guarantees that their implementation is *uniquely defined* and implements the compositions described in Section 5.1. Taken together, challenge **C2** is addressed by SynQ.

5.3. Two interpretations of SynQ models

We now demonstrate two interpretations (software and code generation) of SynQ models which are used in this paper.

[2] Note that `pure` is a generalisation of the composition illustrated in Fig. 5(b), as it allows combinational components, instead of merely streams, to be put alongside sequential components.

```
<program> ::= 'input' <variable_list>
              'output' <variable_list>
              'state' <variable_list>
              'statement' <statement_list>
<variable_list> ::= 'Empty'
                  | <variable_list> <variable>
<variable> ::= <string> ':' <type>
<type> ::= '()'
         | 'BV' <number>
         | 'Prod' <type> <type>
```

```
<statement_list> ::= 'Empty'
                   | <statement_list> <statement>
<statement> ::= <variable> '=' <expression>
              (* set the state on lhs *)
              | '!'<variable> '::=' <variable>
              (* get from the state on rhs *)
              | <variable> '<<=' '!'<variable>
<expression> ::= 'constant' <number>
               | <variable> '+' <variable>
               (* combinational atomic components *)
               | ...
```

Fig. 10. The TAC-like IR employed for code generation.

```
input :  x_0: BV 8
output:  x_4: BV 1
state : !x_5: BV 8
x_1 : BV 8 <<= !x_5: BV 8
!x_5: BV 8 ::=  x_0: BV 8
x_4 : BV 1   =  x_1: BV 8 <U x_0: BV 8
```

Fig. 11. The generated IR code of `isRising` (Fig. 3b).

Software interpretation. The software interpretation interprets SynQ terms into functions in Idris2. That is, SynQ models of type `comb a b` and `seq s a b` are respectively interpreted into functions of type `a -> b` and `Mealy s a b`. Intuitively, such an interpretation is made by replacing all the components a model refers to with their implementations of the corresponding types. In practice, this is conducted by substituting `comb` and `seq` with the following type constructors, which respectively wrap `(->)` and `Mealy`:

```
data Eval: Type -> Type -> Type where
  MkEval: (eval: a -> b) -> Eval a b
data EvalS: Type -> Type -> Type -> Type where
  MkEvalS: (1 evalS: Mealy s a b) -> EvalS s a b
```

For instance, the aforementioned `foo` can be interpreted explicitly by assigning `Eval` to `comb`, which can be written as `foo {comb=Eval}`. The result is a term of type `Eval a b`, which can be pattern-matched to obtain the function of type `a -> b`, i.e., the software implementation of `foo`.

Interpretation for code generation. The current implementation of SynQ allows the LLVM intermediate representation (LLVM IR) [51] and Verilog HDL code to be generated from SynQ models. This respectively allows native software and hardware implementations to be obtained from the same SynQ model. Code generation interprets SynQ models into a TAC-like IR (Fig. 10), which is then translated into target code.

For code generation, the following `TACSeq` type is employed, and combinational components are interpreted by first lifting them to the sequential layer with a unit state.

```
data TACSeq: Type -> Type -> Type -> Type where
  MkTACS: (1 _: (1 _: !* Nat)
       -> LC (!* Nat) (TACSt TACData -> TAC))
       -> TACSeq _ _ _
```

In this type, `TAC` refers to programs in TAC-like IR. As an example, Fig. 11 illustrates the IR code generated from `isRising` shown in Fig. 3b. By abstracting state variables (of type `TACSt TACData`) out, we model states as the context in which the code in the IR is generated. Meanwhile, the linear term of the type `!* Nat` forms the naming context, which guarantees that distinct variables are associated with unique names.

Note that, in the SynQ model of `isRising`, the state is referred to by `get` relatively early. When it is interpreted with `TACSeq`, however, the state variable is assigned to the identifier `x_5`, which suggests that the state variable is the last variable generated in the code. This demonstrates that SynQ model behaviour is decoupled from Idris2's semantics, as the order of variable generation in the generated code differs from the order of state references in the model.

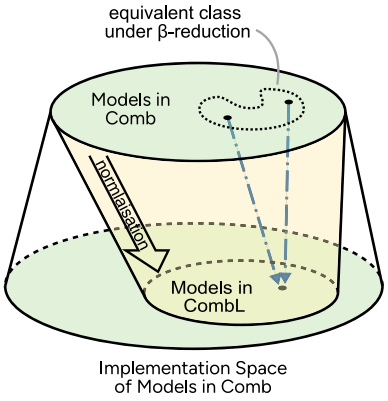


Fig. 12. The visualisation of the normalisation transformation, in which dashed arrows indicate how individual models in `Comb` are transformed into `CombL`.

6. Verifiable transformations for SynQ models

So far, our discussion has focused on how challenges **C1** and **C2** are addressed by leveraging QTT and the tagless final embedding. This consequently allows SynQ to be properly embedded and its interpretations (implementations) to be defined on a closed subset of Idris2 terms. In this section, we demonstrate with two transformations on combinational SynQ terms, namely normalisation and balance-tree, how transformations can be defined and reasoned about in Idris2. These two transformations are representative and correspond to two orthogonal directions in a design process. The normalisation transformation shows how a *vertical* transformation that *lowers* the abstraction level can be introduced and reasoned about. Meanwhile, the balance-tree transformation illustrates the implementation and verification of *horizontal* transformations that are often employed for optimisation purposes, which also shows the feasibility of employing algorithms for transformations.

6.1. Normalisation: a vertical transformation

The normalisation transformation converts applications of SynQ terms using the constructors `lam` and `app` into *direct* applications of glue/atomic components as Idris2 functions. That is, it performs the following transformation on combinational terms in SynQ

`app (lam f) x` $\longmapsto$ `f x`

which eliminates paired occurrences of app and lam and performs β-reduction *before* a SynQ term is interpreted. That is, normalisation maps β-equivalent SynQ terms to a single canonical form, simplifying comparison and enabling further optimisations.

As illustrated in Fig. 12, the normalisation transformation is implemented as a function that takes symantics built upon the set of glue components Comb to symantics constructed by another set of glue components, namely CombL (combinational less), that does not consist of the app constructor. In this way, it is ensured that the result symantics after the transformation cannot consist of any paired app and lam, which then partially ensures the correctness of the transformation. The rest of the correctness is defined in the following proposition, in which $\mathcal{N}$ denotes the normalisation transformation:

Proposition 5. *For all (symantics of) terms t in the combinational layer of SynQ, $\mathcal{N}(t)$ is the symantics of a term in the language defined by CombL and Primitive; and t and $\mathcal{N}(t)$ have the same functional behaviour.*

This proposition is twofold. Firstly, this transformation can be applied to *all* terms, suggesting that $\mathcal{N}$ should be a *total function*. Secondly, $\mathcal{N}$ should preserve the functional behaviour. We demonstrate how the normalisation transformation can be implemented and to what extent these two properties can be asserted. To make the transformation explicit, in this subsection, we desugar SynQ terms by the *dictionary passing* technique [52], which replaces interfaces with records of functions. For instance, the component foo in Section 5.2.1 is distilled to the following:

```
foo: (0 comb: _)
  -> (glues: Comb comb) -> (prims: Primitive comb)
  -> comb a b
```

This allows us to directly refer to component sets as variables, i.e., glues and prims, in the body of foo. It helps distinguish different symantics in the implementation of the normalisation transformation.

6.1.1. A total implementation of normalisation

The implementation has two steps: identify the pattern app (lam f) x in a Comb term, then replace it with direct application in a CombL term. Recall that our intention is to introduce this transformation as a function in the same language in which SynQ is embedded (challenge **C3**); there is no way for us to directly access the ASTs of SynQ terms. To achieve pattern identification, an intermediate type is introduced, which is intended to be used similarly to the Eval type in Section 5.3, and partially interprets SynQ terms to their structure. To guarantee totality, the intermediate type is layered, which allows values (captured by V) to be distinguished from terms (captured by E).

```
data V: (Type -> Type -> Type)
     -> Type -> Type where
  Val: comb () a -> V comb a

data E: (Type -> Type -> Type)
     -> Type -> Type -> Type where
  F: {notUnit: NotUnit a} -> ...
     -> (V comb a -> V comb b) -> E comb a b
  C: V comb a -> E comb () a
```

Here, E comb forms the interpretation of SynQ terms. The separation of values and functions allows the data constructor F of the following form to be avoided:

```
F: {notUnit: NotUnit a} -> ...
   -> (f: E comb () a -> E comb () b) -> E comb a b
```

In which f involves a negative occurrence of E (as the parameter of f), which may lead to a non-terminating pattern-matching on E.

With these types defined, we then define the function that specifies how glue components in Comb are implemented by components in CombL, in which the *second step* is applied. In the desugared form, this function maps a component set to another, which is defined as:

```
impl: {0 comb: _} -> (cl: CombL comb)
   -> (Comb $ E comb)
impl cl = MkComb lam' app' (lift2 cl.prod) ...
```

The second step is then invoked in the definition of app':

```
app': {0 comb: _} -> E comb a b -> E comb () a
   -> E comb () b
app' (F f) (F {notUnit = (NUnit prf)} g) =
  case prf Refl of _ impossible
app' (F f) (C x) = C (f x)
app' (C x) y = C x
```

where the usage of contracts (notUnit = (NUnit prf)) filters out possible non-terminated branches in the definition. By putting them together, the normalisation is wrapped in the following function, for which totality can be formally guaranteed:

```
norm: ... -> (t: {0 comb:_}  -> Comb comb -> comb a b)
  -> ({0 comb':_} -> CombL comb' -> comb' a b)
norm t cl = abst cl.lam (t (impl cl))
```

in which abst maps unreduced lam constructors to CombL.

It is the tagless final approach that allows us to separate the intermediate type into two. The intermediate type employed here maps terms to their *initial* form, in which abstract syntax trees (ASTs) of terms are represented by data. In this way, we can identify patterns and conduct transformations on detected patterns. A sound combination of such an initial embedding and the higher-order abstract syntax (HOAS) [50] used by SynQ is a non-trivial problem [53–55]. Starting from an initial embedding would require either a more complex type system with the S4 modality [56], or a less expressive object language. However, with the tagless final approach, the initial embedding is used only intermediately and internally. This separation will not reduce the expressiveness of SynQ.

6.1.2. The correctness of normalisation

Although the tagless final approach eases the implementation of normalisation, it prevents a general proof of behaviour preservation. To make it clear, we may write the type-encoded (general-case) proposition as

```
prop: ...-> (t: {0 comb:_} -> (Comb comb) -> comb a b)
  -> (evalInterp $ t eGlue)
       = (evalInterp $ (norm t) eGlue')
```

where eGlue and eGlue' are of type Comb Eval and CombL Eval, respectively. These are the instances of component sets Comb and CombL. The function evalInterp: Eval a b -> (a -> b) extracts the Idris2 function from interpreted SynQ terms. The proposition prop states that for all t in Comb, the functional behaviour obtained by directly interpreting t (the left-hand side of =) should be equal to the functional behaviour obtained by interpreting the normalised term (the right-hand side). This type entails a for all quantifier on a function (the symantics t) and hence cannot be proved within Idris2.

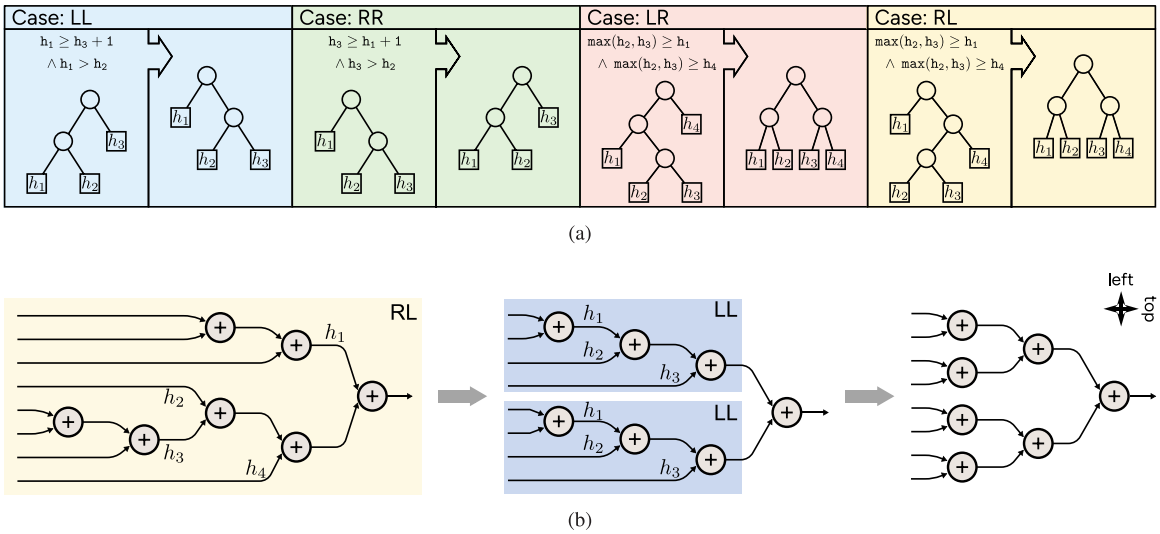
Even without a general proof over all terms, we can still guarantee the correctness of normalisation. In dependent types, we can have a function that generates the desired type/proposition to be proved for each term:

```
contract: ...
    -> (t: {0 comb:_} -> (Comb comb) -> comb a b) -> Type
contract t = (evalInterp $ t eGlue)
               = (evalInterp $ norm t eGlue')
```

Even though this function just produces the proposition without any proof, the normalisation is simple enough to allow the auto tactic to be used, which can generate the proof on-the-fly on demand. For instance, if a proof that a system, namely sys, preserves its functional behaviour after the normalisation is required, the proof can be introduced and used as follows.

```
f: ... -> {auto prf: contract sys} -> ...
f ... {prf} = rewrite prf in ... -- the use of prf
```

In this way, each application of normalisation can be guaranteed to be correct.



(a)

(b)

Fig. 13. An illustration of the balance-tree transformation. (a) Rewriting rules involved in the transformation (LL, RR, LR, and RL). Every rule is defined by a pattern to be matched (left) and the result of applying the rule on the matched pattern (right). Every pattern to be matched consists of the *shape* of a *full balance tree* in which a circle represents a node of the tree and a box denotes a subtree, and *constraints* on subtrees' height (here we overload h_i as both the identifier of a subtree and the subtree's height). (b) An illustration of how a process network (SynQ term) that sums up eight inputs is transformed by iteratively applying rewriting rules (with processes as nodes and sub-networks as subtrees) defined in (a).

Algorithm 1 The algorithm realising the balance-tree transformation, where `rewriteLL` etc. refer to the rewriting rules defined in Fig. 13(a).

```
Require: tree : full binary tree
  function Step(tree)
      if ¬isLeaf(tree) then
        if shape(tree) = LL then
              tree ← rewriteLL(tree)
        else if shape(tree) = RR then
              (* Same for the cases LR/RL. *)
              …
        else
              (* No Match *)
              tree.left ← Step(tree.left)
              tree.right ← Step(tree.right)
        end if
      end if
      return tree
  end function

  repeat
    tr ← Step(tree)
  until isBalanced(tree)
```

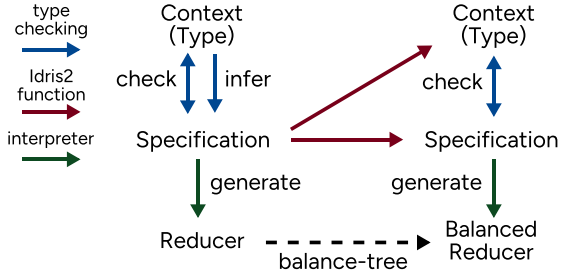


Fig. 14. An illustration of our implementation of the balance-tree transformation. The dashed arrow is the observed transformation, and the red arrows are what we implemented.

6.2. The balance-tree transformation: a horizontal transformation

Besides *vertical transformations* that can be defined as transformations between the underlying sets of components, such as the normalisation, system design also involves transformations that allow us to "optimise" the system under design *before* moving to the abstraction level. We now illustrate with the *balance-tree transformation* how such a *horizontal transformation* can be defined and reasoned about.

6.2.1. Overview of the balance-tree transformation

A *reducer* is a full binary tree whose leaf nodes hold data of the same type and whose internal nodes represent the same binary operation. For reducers with associative binary operations, the balance-tree transformation replaces such a tree with a balanced one of the same leaf count. The minimal height of balanced trees makes the balance-tree transformation an optimisation that *decreases* the length of critical paths in hardware designs and *increases* inherent (data-)parallelism/schedulability for software designs.

An iterative algorithm, as illustrated in Algorithm 1, realises the transformation. For a given tree, the algorithm tries to iteratively match and rewrite the top-most subtree with one of four rewriting rules illustrated in Fig. 13(a), until the tree is balanced. Fig. 13(b) demonstrates how, with the algorithm, a reducer that sums up eight inputs is transformed into a balanced reducer.

In the rest of this section, we demonstrate how Algorithm 1 can be implemented as an Idris2 function for SynQ terms by utilising types and how the correctness of such a function can be reasoned about in Idris2. Note that our current implementation only partially implements the balance-tree transformation: Our implementation of the transformation skips the step that detects reducers from SynQ terms and starts with a given, well-specified reducer with an arbitrary structure. The correctness of the balance-tree transformation is twofold: the transformation should produce a reducer with minimal height and preserve the functional behaviour of the original reducer. We only show behaviour preservation here since the minimal-height proof depends on a critical-path interpretation not included in this paper.

6.2.2. The implementation of balance-tree transformation

An overview of our implementation of the balance-tree transformation is illustrated in Fig. 14. The implemented transformation takes

the specification of a reducer, applies Algorithm 1, and produces the new context and specification with which the balanced-reducer can be generated. The application context here refers to the input/output type expected by the reducer, which, as will be discussed later, determines the structure of the reducer. Two key pillars of the transformation are *the specification of reducers* inferred from context and type-checked for correctness, and *the type- and specification-level transformation*, which preserves well-typedness and guarantees correct usage of the transformed reducer. They guarantee that local application of the transformation to a subsystem always yields a well-formed system.

Inferable and general specification of reducers. From the most general view, a reducer in SynQ is a component, named `reduce`, of the following type

```
reduce: ... -> (f: comb (a, a) a) -> comb as a
```

in which `f` is the binary operation applied by the reducer, and `as` is the type of a collection of data of type `a`. The type `as` and `a` form the application context of `reduce`. To make it general, `as` is expected to be the type variable that can be substituted by any type that a reducer can be applied to. For instance, if `UInt8` is the expected output type (`a`), `as` can be either `UInt8` or `((UInt8, UInt8), UInt8)` but not `(BitVec 9, UInt8)`. To this end, we employ a type-encoded contract (similar to `{auto aIsSig: Sig a}` explained in Section 5.2.3) for specifying `as`. The definition of this contract is twofold, as illustrated below:

```
OfType: Type -> Type -> Type
OfType x y = x = y

data All: (pred: Type -> Type) -> Type -> Type where
  AllU: {x: Type} -> (pred x) -> All pred x
  AllP: All pred a -> All pred b -> All pred (a, b)
```

`OfType a b` forms the predicate that two types `a` and `b` are equal and `All pred as` asserts if all parts of `as` satisfy the predicate `pred`. The reducer is then of the type

```
reduce: ... (f: comb (a, a) a)
  -> {auto all: All (OfType a) as} -> comb as a
```

In this type, `all` is now the witness of that `as` is a type whose data can be reduced. Since `All` is constructed by a simple induction, we can use the `auto` keywords to leverage the type checker to *infer* the value of `all`.

With the type of `reduce` given, it can be implemented as follows:

```
(<<): ... -> comb b c -> comb a b -> comb a c
(<>): ... -> comb a c -> comb b d -> comb (a, b) (c, d)

reduce f {all = (AllU Refl)} = lam id
reduce f {all = (AllP p1 p2)} =
  f << (reduce f) <> (reduce f)
```

in which `(<<)` and `(<>)` are the sequential and parallel composition, respectively. This implementation first matches the form of `all`, which consequently allows its input type `as` to be matched, and then inductively constructs the reducer. In this way, the structure of the reducers generated by `reduce` is deterministically dictated by `as`. For instance, `reduce f` in a context where `as=((a, a), a)` will produce a reducer that first applies `f` to reduce its *first* two inputs. Whereas, in a context with `as=(a, (a, a))`, the reducer produced will first apply `f` to reduce its *last* two inputs.

Note that the implementation of `reduce` introduces a one-to-one correspondence between the "shape" of its input type, which is introduced by the value of `all`, and the structure of the reducer it generates. It reduces the design space in the context where `reduce` can be applied because it always introduces a reducer with a specific structure in a given context. However, the lost design space is retrieved by introducing transformations that preserve functional behaviour, e.g., the balance-tree transformation introduced in this section.

The transformation. With the one-to-one correspondence introduced between a reducer generated by `reduce` and its application context, the expected behaviour of the transformation implemented at the type and specification level is that: Given a witness `allA: All (OfType a) as` from which a reducer `comb as a` can be generated, the transformation produces a new context where a reducer `comb as' a` can be generated, with `as'` balanced and `allA': All (OfType a) as'` derivable. We may consider that this transformation *generates adapters* that enable balanced reducers to be used in unbalanced contexts. Only in our case, a generated adapter determines the balanced reducer to be used, instead of the other way around.

Generating adapters before knowing the reducer is non-trivial, as the input *type* of the balanced reducer must be computed first to determine the reducer itself. In the dependent type system, we can use the dependent pair, which was introduced in Section 3.1, to accomplish it. A dependent pair, denoted in Idris by `(x ** y)`, is of the type `(x: a ** a -> Type)`, i.e., the *type* of its second element is determined by the *value* of its first element. With this type, the balance-tree transformation is of the following type:

```
balance: ... => {default 20 max_iter: Nat}
  -> {auto allA: All (OfType a) as} -> comb () as
  -> (as': Type
       ** (comb () as', All (OfType a) as', Sig as'))
```

For a given signal, the transformation first computes the balanced type `as'` and then the transformed signal of this type and contracts on this type based on Algorithm 1. Different from Algorithm 1, the number of iterations this function can be invoked is limited by the parameter `max_iter`. In this way, asserting the totality and formalising the desired property of this function is easier.

Note that our specification and implementation of `reduce` and `balance` do not require that the binary operation is associative. Indeed, the associativity requirement is introduced only in the specification of the behaviour-preservation property, which we discuss later. This is because associativity may not be a trivial property whose proof can always be derived by the type checker. Avoiding it at the modelling level enables us to use `reduce` and its transformation more flexibly, e.g., in contexts where the binary operation is a black box whose associativity cannot be formally proved. Meanwhile, the proof of the behaviour-preservation property presented in the next section is sufficient to guarantee the correctness of this transformation, which serves as a reliable contract whenever formal reasoning is required.

6.2.3. The correctness of the transformation

Based on `reduce` and `balance` introduced earlier, the property expected for the balance-tree transformation is stated as follows, which directly corresponds to a type in Idris2:

Proposition 6. *Let `f` be the symantics of a binary operation with inputs and output of type `a`, `as` be a type such that a term `allA` of type `All (OfType a) as` can be constructed, and `xin: comb () as` be an input of type `as`. For all `k` and `(as' ** (xin', ...)) = balance {max_iter=k} xin`, applying `reduce f` on `xin` and applying `reduce f` on `xin'` result in the same functional behaviour (Idris2 function).*

This proposition is proved by induction over `k`, using the lemma that rewriting rules illustrated in Fig. 13(a) preserve the functional behaviour of reducers under the condition that `f` is associative. The latter lemma is proved by equational reasoning in Idris2, in which the left-hand side of a rule in Fig. 13(a) is transformed to its right-hand side by utilising the equivalence relation introduced by associativity, as illustrated in Fig. 15.

Unlike the case of normalisation, whose correctness cannot be directly proved due to the quantifier on symantics occurring in the specification, it was possible for us to obtain a complete proof in Idris2 about the correctness of the balance-tree transformation. This is

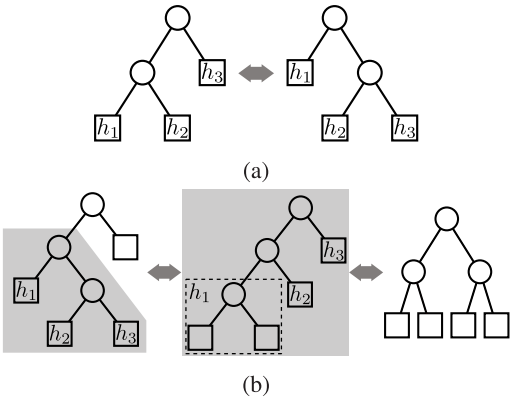


Fig. 15. An illustration of how we utilise associativity to prove that the rewriting rule LR in Fig. 13(a) preserves the functional behaviour. (a) The equivalence relation introduced by associativity. (b) Application of the relation (in the shadowed region) to introduce the equivalence between the lhs and rhs of the LR rule.

because the balance-tree transformation operates on specifications that *generate* symantics of reducers, rather than on their symantics directly. In this way, we need not reason about the desired property based on the decomposition of symantics and hence avoid the issue discussed in Section 6.1.2.

7. Examples

We now demonstrate, with three examples, how SynQ and transformations defined on it facilitate the design and verification of synchronous systems:

- a *direct-sampling FM (frequency modulation) receiver* modelled in SynQ, and implemented by interpreting the model to its software and Verilog implementations, which highlights how systems can be modelled in SynQ with hierarchies and, also, how high-level features of Idris2, such as elaborator reflection [14], can be used for system modelling;
- a *FIFO buffer with a simple handshake protocol*, a fundamental component in signal processing applications, to showcase how SynQ allows leveraging Idris2 for testing and formally verifying synchronous systems;
- *direct-form FIR filters* modelled in SynQ to illustrate the usage of transformations.

7.1. Direct-sampling FM receiver

In direct radio frequency (RF) sampling architecture [57], the RF signal is directly sampled and processed in the digital domain without down-conversion to an intermediate frequency (IF). In this example, a direct-sampling FM receiver, as shown in Fig. 16, is modelled using SynQ, simulated via its software interpretation, and deployed on an FPGA using the generated Verilog HDL code. The simulation is employed to validate the functionality of the model (and sub-models) during the design phase. For instance, the DDC (digital down-converter) frequency response is evaluated at 8192 points over the full input bandwidth and 1024 points over the channel bandwidth (Fig. 17). This validates the DDC's channel selectivity and passband flatness. Simulation-based validation is justified by the system's computational intensity and the tight relation between different implementations of the same SynQ model (Section 5.2.3). The deployed system is able to process RF signals sampled at 122.88 MHz to produce a demodulated baseband signal at 960 kHz in real-time, i.e., it guarantees an output sample for every 128 input samples, subject only to deterministic latency. Fig. 18 illustrates the spectrum of a demodulated signal produced by the FM receiver when selecting the channel with a centre frequency of 26.68 MHz (96.2 MHz[3] in the 2nd Nyquist zone of the sampled signal). The observation of all components of an FM stereo broadcast signal validates the functionality of the deployed design.

7.1.1. The top-level SynQ model

The FM receiver is modelled in SynQ as a hierarchy of components. The architecture of the FM receiver is illustrated in Fig. 16(a) with a partially shown hierarchy. As illustrated, the top-level model consists of four components: two DDCs, which, together, perform quadrature down-conversion of the received RF signal to select 180 kHz bandwidth channels; a component that calculates the phase of the down-converted signal; and a component that calculates the first-order difference of the phase to produce the demodulated signal. Drilling down further, each DDC is also modelled as a hierarchy of components, including a mixer, a four-stage cascaded integrator-comb (CIC) decimator with a decimation factor of 128, and a 64-tap compensation filter.

The corresponding top-level model in SynQ is shown in Fig. 16(b). Its type declaration follows the same pattern as the `isRising` shown in Fig. 3b and the discussion in Section 5. That is, lines 1 to 5 declare the component sets (constraints over `comb` and `seq`) employed, in which lines 2 to 5 correspond to resources like components bound with multiplicity 1, and lines 6 and 7 specify types of the input, output, and internal state of the system. Note that, in line 1, an extra interface, `Mult18`, is introduced, which consists of the following component:

```
mult18: comb () (BitVec 18) -> comb () (BitVec 18)
     -> comb () (BitVec 36)
```

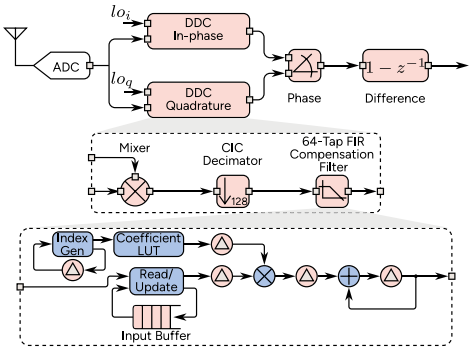
This component abstracts, in a very compact way, the DSP48E1 slice[4] in the target FPGA, and enables it to be utilised for an efficient implementation. Meanwhile, the resource-like components, such as `ddcIRegs` and `catan2Regs`, are not of the type of `Reg a comb seq`, which is different from the declaration of `isRising`. Indeed, a component like `ddcIRegs` refers to *all the resources required by the in-phase DDC component*, which is internally defined via a hierarchy of components of type `Reg a comb seq`, but treated as a black box in the top-level model.

From line 8 to line 17 is the definition of the top-level model. Lines 11 to 13 instantiate the four components mentioned above from the resources declared in lines 2 to 5. Restricted by multiplicity 1, the four components are the only sequential components that can be instantiated inside the top-level model, which, in the meantime, must be used to construct the top-level model. This facilitates resource tracking from the top-level model.

From line 14 to line 17, the top-level model is constructed. Here, glue components at the sequential layer enable an interesting pattern for model composition. Specifically, recall from Section 5.1 that stateful components (Mealy machines) specified by their single-step behaviour may be sequentially composed either by merely connecting their inputs and outputs, as shown in Fig. 5(a), or by sharing their state, as shown in Fig. 6(a). The grid-like arrangement shown on the right-hand side of `(<-)` in lines 14 to 16 of Fig. 16(b) (separated by `(<<<)`) corresponds to components composed in the way shown in Fig. 19. That is, a component in the grid is composed with its horizontal adjacent components by connecting their inputs and outputs, and is composed with its vertical adjacent components by sharing its state. This grid-like composition, together with two special components, `pass`, which simply passes through its input and state, and `nop`, which produces nothing to its output and passes through its state, enables the top-level model to be constructed in a way that closely resembles the architecture illustrated in Fig. 16(a).

[3] The Sveriges Radio P2 channel at authors' location.

[4] https://docs.amd.com/r/en-US/ug953-vivado-7series-libraries/DSP48E1.



(a)

```
fmDemod: (Seq comb seq, Primitive comb, Mult18 comb)
      => (1 ddcIRegs      : DDCRegs comb seq)
      -> (1 ddcQRegs      : DDCRegs comb seq)
      -> (1 catan2Regs    : CAtan2Regs comb seq)
      -> (1 diffRegs      : UnwrapDiffRegs comb seq)
      -> seq FMDemodSt (BitVec 16, BitVec 18, BitVec 18)
                       (BitVec 1, BitVec 32)
fmDemod ddcIRegs ddcQRegs catan2Regs diffRegs =
  abst $ \xin =>
    let (dIn, loI, loQ) = unpack xin
        ddcI # ddcQ     = (ddc' ddcIRegs) # (ddc' ddcQRegs)
        atan2           = cordicAtan2' catan2Regs
        diff            = unwrapDiff' diffRegs
    in do ifI <- pass <<< pass  <<< pass         <<< ddcI loI dIn
          ifQ <- pass <<< pass  <<< ddcQ loQ dIn <<< nop
          res <- diff <<< atan2 <<< zip ifI ifQ  <<< nop
          pure res
```

(b)

Fig. 16. FM receiver: (a) illustration with a partially shown hierarchy; (b) top-level definition in SynQ.

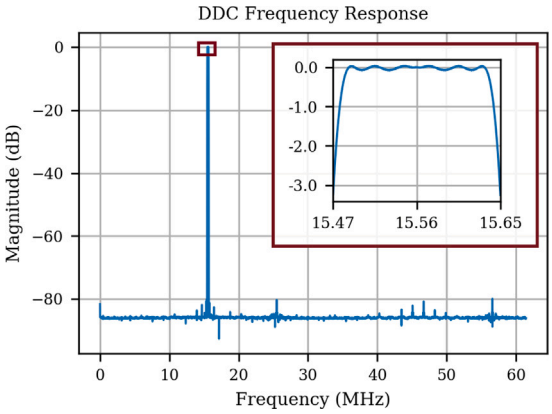


Fig. 17. The frequency response of the digital down-converter (DDC) obtained from the software implementation of the FM receiver.

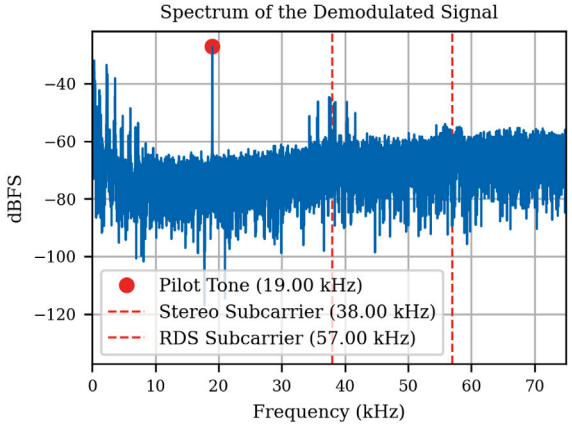


Fig. 18. The spectrum of the demodulated signal received by the FM receiver with local oscillator (LO) tuned to 26.68 MHz (96.2 MHz in the 2nd Nyquist zone).

7.1.2. Modelling components in Idris2

So far, our discussion has focused on components built with glue and atomic components defined in SynQ. By keeping models in SynQ a closed subset (challenge **C1**, Section 5.2), every Idris2 function whose result type is `comb a b` or `seq s a b` returns a valid term in SynQthat can be used as a component in SynQ models. This enables the expressiveness of Idris2 to be leveraged to extend the expressiveness of SynQ. We illustrate this with two subcomponents of the FM receiver:

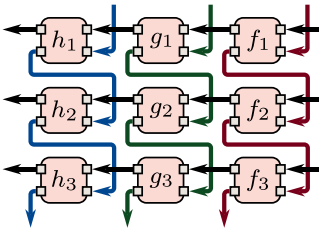


Fig. 19. The grid-like composition of components in which a component is composed with its horizontal adjacent components by connecting their inputs and outputs, and is composed with its vertical adjacent components by sharing its state.

the integrator stage of the CIC decimator and the coefficient lookup table (LUT) of the compensation filter.

The integrator stage. The integrator stage of the CIC decimator consists of a chain of accumulators, in which each accumulator's output is truncated to a certain bit width, which is carefully selected so that subsequent stages can compensate for any overflow caused by truncation, thereby enabling a more resource-efficient implementation [58]. Consequently, the CIC decimator's correctness relies heavily on correctly handling this truncation, which is chosen ad hoc based on the input/output bit width and number of stages.

Conventionally, in a language like Verilog HDL, the number of least significant bits (LSBs) to be truncated may be given as a parameter to the module of each accumulator. This practice hard-codes the truncation at the integrator stage level, hindering adaptive reuse. Modern embedded languages like Chisel (in Scala) can generate a chain of accumulators from a data structure specifying each accumulator's truncation. By leveraging the expressiveness of QTT, SynQ enables the bit width of each accumulator to be explicitly specified in the type of the integrator stage component, and the parameters of the truncations involved to be automatically calculated based on the given type. This enables a correct configuration to be specified, generated, and verified by type checking. For instance, the integrator stage in the FM receiver is given as the follows:

```
cicInt4: (Seq comb seq, Primitive comb)
      => (1 regs: CICIntRegs [44, 37, 31, 26] comb seq)
      -> seq (CICIntSt [44, 37, 31, 26])
             (BitVec 44) (BitVec 23)
cicInt4 (r1 # r2 # r3 # r4) =
  acc r4 <<< acc r3 <<< acc r2 <<< acc r1
```

As shown, the bit width of each accumulator is explicitly specified by `[44, 37, 31, 26]`. Meanwhile, the system is specified as four sequentially composed accumulators without any hard-coded parameters for the truncation. This is achieved by leveraging the dependent type fragment of QTT. Specifically, the type of `acc` is given as follows:

```
acc: (Seq comb seq, Primitive comb)
  => {iWidth: Nat} -> {oWidth: Nat}
  -> (1 reg: Reg (BitVec iWidth) comb seq)
  -> seq (!* BitVec iWidth) (BitVec iWidth) (BitVec oWidth)
```

The input and output bit widths (`iWidth` and `oWidth`) are both part of the type signature of `acc`, and the (implicit) parameters of `acc`. This enables the values of `iWidth` and `oWidth` to be first inferred by type checking based on the given type of the integrator stage, and then used to calculate the parameters of the truncation inside `acc`.

The coefficient LUT. The compensation filter in the FM receiver is a 64-tap FIR filter implemented in sequential form. Such a filter consists of a coefficient generator that produces the corresponding coefficient based on the current state by indexing into the coefficient LUT. The coefficient LUT should be parametrised during design to easily evaluate different coefficient sets, but fixed in the final implementation to minimise execution overhead. This is achieved by employing the elaborator reflection [14] to generate the coefficient LUT at the type checking stage.

An elaborator reflection process in Idris2 is defined as a term of type `Elab a`, which is known as an elaborator script. Such an elaborator script instructs the elaborator (a pass of Idris2's type checking process) to generate a term of type `a` and insert it into the program at the place where the script is invoked. This allows all intermediate values and computations involved in an elaborator script to be executed at the type checking stage with only the final result being inserted into the program. In the FM receiver, the coefficient LUT is generated by an elaborator script of the following type:

```
lutGen: (Primitive comb)
  => (Vect numEle $ BitVec dataWidth)
  -> Elab $ (comb () $ BitVec idxWidth)
  -> comb () (BitVec dataWidth)
```

It takes a vector of coefficients and returns an elaborator script that generates the coefficient LUT of the form assume a 6 bits index is used:

```
coefLut idx =
  mux21 (slice 5 6 idx)
        (mux21 (slice 4 5 idx) ...) -- idx = 0b1xxxxx
        (mux21 (slice 4 5 idx) ...) -- idx = 0b0xxxxx
```

In this form, there is no intermediate computation at run time, and the coefficients are indexed by bits of the input index, rather than comparisons between values, which is efficient for hardware implementation. By leveraging elaborator reflection, we can generate coefficient LUTs for arbitrary sets of coefficients (with possibly different sizes) that can be efficiently implemented.

7.2. FIFO buffer with handshake

The handshake protocol to be implemented for the FIFO buffer is illustrated in Fig. 20. This protocol has a forwarded data signal (sender to receiver) whose elements may include invalid values (denoted by ⊥) that need not be saved, and a backward acknowledgement (ack) signal by which the receiver confirms receipt and readiness for new data. A valid datum is held by the sender until the `ack` signal is received. When the current data is invalid, the receiver may acknowledge readiness for new data. In such cases, the `ack` signal is held until a valid datum is received. Note that data receiving and `ack` production happen simultaneously at the same clock event, resulting in zero-delay feedback, as shown in Fig. 20, in high-level models, which cannot be directly represented in SynQ.

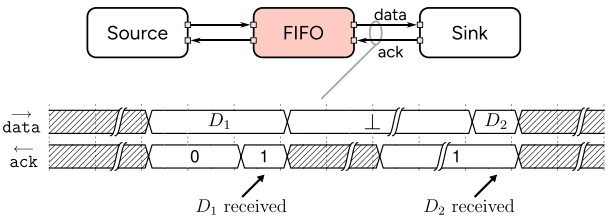


Fig. 20. An illustration of the handshake protocol implemented in Section 7.2.

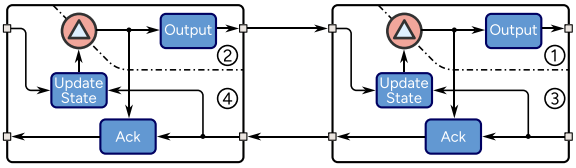


Fig. 21. An illustration of the composition of two FIFO buffers. The dashed lines indicate where a FIFO buffer is partitioned.

7.2.1. SynQ model of the FIFO

To efficiently model the state of a FIFO buffer with arbitrary depth, a vector-like type `Repeat n a` shown as follows is introduced:

```
Repeat: Nat -> Type -> Type
Repeat 0 t = ()
Repeat 1 t = t
Repeat (S $ S k) t = (t, Repeat (S k) t)
```

Based on this type, we can have a universal component inductively defined for updating the state of an arbitrary FIFO. Its core is the following component that updates the state.

```
dropLast: ... -> {n: _}
  -> comb () (Repeat (n+1) a) -> comb () (Repeat n a)
dropLast {n = 0} x = unit
dropLast {n = (S 0)} x = proj1 x
dropLast {n = (S (S k))} x = prod (proj1 x)
                                  (dropLast $ proj2 x)
```

The feedback loop is eliminated by partitioning handshake protocol models into two parts corresponding to the forward (data) and backward (ack) paths. This partitioning is enabled by Idris2's higher-order functions and SynQ's two-part delay element. Two partitioned FIFO buffer models and their composition are illustrated in Fig. 21. Following the linearity restriction applied to `set`, QTT guarantees that sub-models 3 and 4 in Fig. 21 are also restricted to be used exactly once and hence guarantees that the partitioning is aligned with SynQ's abstraction. With this partitioning, the delay is inserted between sub-models 1 and 3 (and 2 and 4), which breaks the zero-delay feedback loop internally and allows the FIFO buffer to be modelled in SynQ.

7.2.2. Property-based testing and formal verification

Interpreting SynQ models as Idris2 functions allows Idris2 to be leveraged for property-based testing and formal verification. For example, consider a desired property of a FIFO buffer that: *all valid data received by a FIFO buffer will, if required by its successor, be produced in the order it was received*. We can easily define the following Idris2 function, in which the software implementation of a FIFO buffer (`fifoMealy`) is tested against the property under the dataflow semantic model.

```
fifoProp input =
  let (dataIn , ackIn)  = unzip input
      (dataOut, ackOut) = unzip $ fifoMealy input
  in (squeeze $ dataOut `zip` ackIn)
       <: (squeeze $ dataIn `zip` ackOut)
```

$$\varphi_1 ::= \forall x : \mathrm{BV}_n.\ x \mathbin{\overline{\wedge}} 0 \equiv 0$$
$$\varphi_2 ::= 1_n \mathbin{\overline{\wedge}} 1_n \equiv 1_n$$
$$\varphi_3 ::= \forall x, y : \mathrm{BV}_n.\ x \mathbin{\overline{\wedge}} y \equiv y \mathbin{\overline{\wedge}} x$$
$$\varphi_4 ::= \overline{\neg} 0_1 \equiv 1_1$$
$$\varphi_5 ::= \overline{\neg} 1_1 \equiv 0_1$$
$$\varphi_6 ::= 0_3 \mathbin{\overline{<}} 4_3 \equiv 1_1$$
$$\varphi_7 ::= 0_3 \mathbin{\overline{<}} 5_3 \equiv 1_1$$
$$\varphi_8 ::= 0_3 \mathbin{\overline{<}} 1_3 \equiv 1_1$$
$$\varphi_9 ::= \forall x, y : \mathrm{BV}_3.\ \overline{\neg}(x = 0_3) \equiv 1_1 \implies (x \mathbin{\overline{<}} y) \equiv ((x \mathbin{\overline{+}} 7_3) \mathbin{\overline{<}} (y \mathbin{\overline{+}} 7_3))$$
$$\varphi_{10} ::= 5_3 \mathbin{\overline{+}} 7_3 \equiv 4_3$$
$$\varphi_{11} ::= \forall x : \mathrm{BV}_3.\ (x \mathbin{\overline{<}} 4_3 \equiv 1_1) \implies (0_3 \mathbin{\overline{<}} (x \mathbin{\overline{+}} 1)) \equiv 1_1$$
$$\varphi_{12} ::= \forall x : \mathrm{BV}_3.\ (x \mathbin{\overline{+}} 0_3) \equiv x$$

Fig. 22. Behaviour identified during proving the proposition on a FIFO buffer with depth 4. Overlined operators ($\overline{\square}$) are black-boxes components whose behaviour are not exposed to the type checker. Subscript denotes the width of a constant, i.e., k_m denotes the constant k represented by m bits.

In this function, `zip` and `unzip` send lists of products to products of lists and vice versa; `squeeze` drops events, which consist of an invalid datum or a false `ack` signal, in a given list; and `(<:)` is the *prefix-of* predicate that asserts whether a list on its lhs is a prefix of the list on its rhs. Note that functions invoked in `fifoProp`, except `fifoMealy`, are normal Idris2 functions that may not even be implementable as synchronous systems.

Instead of directly verifying the property that is being tested, in QTT we specify and verify the implemented FIFO's stepwise behaviour. This means that the verification concerns the system's output and state transition for each input event. For instance, the following proposition is specified and verified:

Proposition 7 (*Data Is Stored If* `ack` *Is Set*)**.** *For arbitrary valid input data and* `ack` *signal from the successor stage, if the* `ack` *signal 1 is produced, then, at the next clock event, the current input data should be stored into the specific position (tail) of the FIFO buffer's state.*

The verification follows the conventional theorem-proving process. This minimises QTT's performance limitations in Idris2.

Note that, when treating SynQ's atomic components as black boxes whose behaviour is unexposed to the type checker, proving a proposition of an implemented model reveals the properties these black boxes must satisfy. Fig. 22 illustrates properties identified when proving the above proposition on a depth-4 FIFO buffer, where φ_9 and φ_{11} show that a component's behaviour can be mutually defined with other components rather than characterised directly. These properties serve as *proof obligations* to be verified against the external (C in our case) implementation of these components. For instance, φ_1 suggests the following contract specified in ACSL (ANSI/ISO C Specification Language) [59] for the C function that implements $\overline{\wedge}$:

```
/*@ assigns \nothing;
  @ ensures (val_2 == 0) ==> (\result == 0);
  @ */
uint64_t bv_and(uint8_t len, uint64_t val_1, \
                uint64_t val_2);
```

The verification of this contract then ensures the correctness of the FIFO buffer.

7.3. FIR filter and transformations

We now employ the design of FIR filters as an example to demonstrate how transformations can be used in the design of synchronous systems. Two transformations to be illustrated here are the balance-tree transformation, as described in Section 6.2, and the product-elimination transformation, which follows an implementation similar to the normalisation transformation described in Section 6.1.

7.3.1. The balance-tree transformation

Following the same principle as how FIFOs are modelled, components of an FIR filter can also be inductively defined based on the `Repeat` type. With these components, the top-level SynQ model of FIR filters with an arbitrary number of taps can be given as shown in Fig. 23. Note that, Fig. 23(a) explicitly specifies that the interpretation of `sum (comb')` *need not* be the same as the interpretation (`comb`) of FIR filters. In this way, the space of applying component-set-based transformations, such as the normalisation transformation, is introduced.

The inductive nature of `Repeat` enforces `sum` to generate a sequential reducer (Fig. 23(b)), as `reduce` always decomposes `xs: Repeat (n+1) a` into its head `x: a` and tail `xs': Repeat n a` by induction. This can be suboptimal when implementing the FIR filter on a platform that supports parallelisation. Rather than defining an ad-hoc reducer, the balance-tree transformation requires only modifying line 15 of the original model (Fig. 23(a)) to introduce parallelism:

```
{- same as before -}
lam $ \x => let (_ ** (x', _, _))  = balance x {...}
            in  (reduce adder2) << x'
```

The modified SynQ term then models FIR filters with the balanced sum operation. Meanwhile, with Proposition 6 proved in Idris2, we can easily verify that the transformed model has the same behaviour as the original model by proving that `addder` is associative. As discussed earlier, the proof of associativity can be conducted externally and ad hoc when the number of taps, the width of data, and the width of coefficients have been determined.

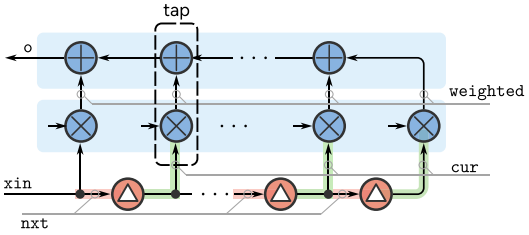
The effect of the balance-tree transformation. The application of the balance-tree transformation yields a better design space when a transformed model (application) is to be mapped to an architecture that supports parallelisation. Following the *Y-chart approach* [8], we demonstrate the transformation's effect by showing that *mapping a transformed FIR filter onto an abstract platform with two execution units (EUs) and unlimited communication is* ***most likely*** *to yield better performance than mapping the original.* The mapping assigns each statement in an IR program to the EUs in the platform, as shown in Fig. 24(a). Two statements assigned to different EUs can be executed simultaneously if all variables on their right-hand side have been produced. Based on this execution model, two performance numbers are calculated for a mapping: the *number of steps* required to execute the program on the architecture and the *number of communications*, which counts the number of times a statement requires data produced from another EU, as illustrated in Fig. 24(c).

The distribution of these performance numbers, obtained from 10,000 randomly generated mappings that map FIR filters with 32, 64, and 128 taps (each) to the aforementioned abstract platform, are illustrated in Fig. 25. As shown in the sub-figures i-iii of this figure, the number of communications required by the original and transformed FIR filters follows similar distributions, while the number of steps required by the transformed FIR filters is on average fewer. Figs. 25(a) to 25(c).iv further show that, for each generated mapping, applying the transformation reduces the number of steps it requires.

It is worth mentioning here that the platform employed here is highly abstract. Hence, the only conclusion that can be drawn is that the balance-tree transformation makes FIR filters more "parallelisation-friendly" in the sense that it introduces a better design space for mapping FIR filters to parallel architectures. Actual performance gains

```
1  mkFIR: {0 comb: _} -> ...
2  mkFIR coefs (MkReg get set) =
3    let {- proofs about the context -}
4    in abst $ \xin =>
5         do cur <- get
6            let cur'     = prod xin cur
7                weighted = multKs {...} coefs cur'
8                o        = sum << weighted
9                nxt      = dropLast cur'
10           _ <- set nxt
11           pure $ o
12 where
13   sum: {0 comb': _} -> ...
14   sum = let {- proofs about the context -}
15         in lam $ \x => (reduce adder2) << x
```

(a)

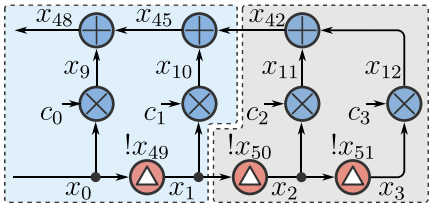


(b)

Fig. 23. The top-level SynQ model of direct style FIR filters with an arbitrary number of taps: (a) the SynQ model; and (b) the visualisation of the SynQ model whose correspondence to the SynQ model is marked with coloured boxes. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

```
P1 x_1: BV 16 <<= !x_49: BV 16
P2 x_2: BV 16 <<= !x_50: BV 16
P2 x_3: BV 16 <<= !x_51: BV 16
P1 x_9: BV 32  = 16'd16384: BV 16 * x_0: BV 16
P1 x_10: BV 32 = 16'd16384: BV 16 * x_1: BV 16
P2 x_11: BV 32 = 16'd16384: BV 16 * x_2: BV 16
P2 x_12: BV 32 = 16'd16384: BV 16 * x_3: BV 16
P2 x_42: BV 32 = x_11: BV 32 + x_12: BV 32
P1 x_45: BV 32 = x_10: BV 32 + x_42: BV 32
P1 x_48: BV 32 = x_9: BV 32  + x_45: BV 32
P1 !x_49: BV 16 ::= x_0: BV 16
P2 !x_50: BV 16 ::= x_1: BV 16
P2 !x_51: BV 16 ::= x_2: BV 16
```

(a)



(b)

P1

```
1: x_1: BV 16 <<= !x_49: BV 16
2: x_9: BV 32  = 16'd16384: BV 16 * x_0: BV 16
3: x_10: BV 32 = 16'd16384: BV 16 * x_1: BV 16
4:
5:
6: x_45: BV 32 = x_10: BV 32 + x_42: BV 32
7: x_48: BV 32 = x_9: BV 32  + x_45: BV 32
8: !x_49: BV 16 ::= x_0: BV 16
```

P2

```
x_2: BV 16 <<= !x_50: BV 16
x_3: BV 16 <<= !x_51: BV 16
x_11: BV 32 = 16'd16384: BV 16 * x_2: BV 16
x_12: BV 32 = 16'd16384: BV 16 * x_3: BV 16
x_42: BV 32 = x_11: BV 32 + x_12: BV 32
!x_50: BV 16 ::= x_1: BV 16
!x_51: BV 16 ::= x_2: BV 16
```

(c)

Fig. 24. The demonstration of a mapping of a 4-tap FIR filter on the given architecture and how the performance numbers are calculated. (a) the mapping of the generated IR program; (b) the illustration of the mapping in (a); and (c) the illustration of how the performance numbers are calculated, in this case, the number of steps required is 8, and the number of communications is 2 (marked by red boxes). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

depend on the concrete platform and mapping strategy. For instance, Fig. 25 shows that the original and transformed FIR filters require a number of steps and communications in the same order of magnitude. Meanwhile, the transformation does not significantly affect the number of communications. Hence, the expected performance improvement shown in Fig. 25 may not be achieved when the communication cost is high, e.g., on a conventional MPSoC (multicore processor system-on-chip) platform. On the other hand, the transformation is expected to have a more significant effect on platforms with low communication cost, e.g., single-core architectures with SIMD (single instruction, multiple data) capabilities.

7.3.2. The product-elimination transformation

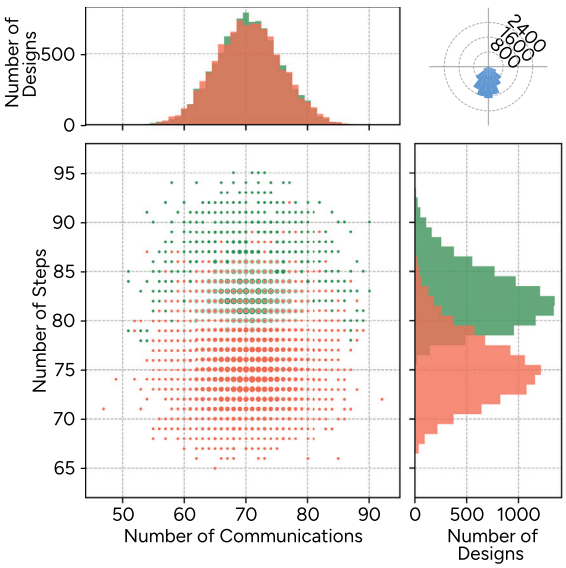
In Section 5.2.3, we discussed that challenge **C2** is partially addressed by employing components, such as `lam` and `abst`, which allow bindings in Idris2 to be borrowed with their behaviour implemented separately. With this approach, the desired behaviour (interpretations) of system models is implemented by using Idris2's behaviour. For some interpretations, such as the one for code generation (Section 5.3), this approach suggests that a trade-off between *simplicity* and *efficiency* of their implementation needs to be taken. For instance, a direct (simple) implementation of the code generation discussed in Section 5.3 will interpret the following SynQ term

```
proj1 (prod (add (const $ BV 1) (const $ BV 1))
            (add (const $ BV 2) (const $ BV 3)))
```

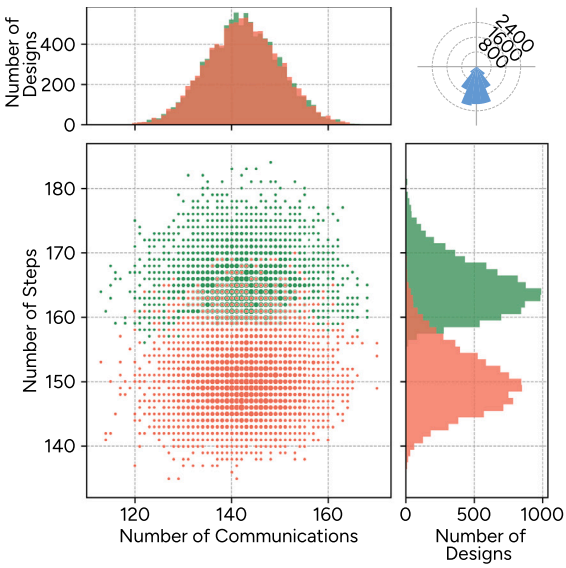
into the following IR program in which the dead code `x1: BV 8 = ...` is also generated:

```
input: (): ()
output: x_0: BV 8
state: !(): ()
x_0: BV 8 = 1: BV 8 + 1: BV 8
x_1: BV 8 = 2: BV 8 + 3: BV 8
```

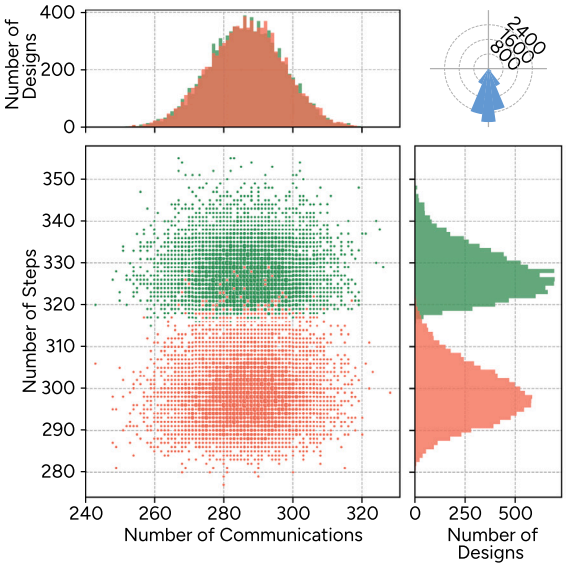
This occurs because code generation processes all sub-components bottom-up and context-independently. This implementation eases the formal reasoning of the interpretation, as each generated code snippet can be reasoned about independently. It is, however, inefficient because code for components that are not eventually used will also be generated, as illustrated above. Even though such dead code can be easily



(a) Results for 32-tap FIR Filters



(b) Results for 64-tap FIR Filters



(c) Results for 128-tap FIR Filters

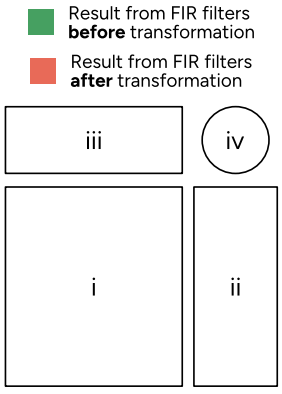


Fig. 25. The distribution of the performance numbers obtained from 10,000 randomly generated mappings of FIR filters with 32, 64, and 128 taps before and after the balance-tree transformation (each). In each sub-figure (a, b, and c), (i) the scatter plot illustrating the overview of the performance numbers obtained from mappings of FIR filters before and after the transformation; (ii) the distribution of the *number of steps* required for the execution of the mapped FIR filter; (iii) the distribution of the *number of communications* required for the execution of the mapped FIR filter; and (iv) the statistic result showing *on which direction* (under the coordinate of sub-figure i) the performance numbers of a mapped FIR filter is affected by the transformation, i.e., the transformation moves the performance numbers of a mapped filter downwards (south) if, for the same mapping, the transformed filter requires *fewer* number of steps and *the same* number of communications.

eliminated based on the IR (Section 5.3), it significantly slows down the code generation, especially when large products are encountered. For example, when the balance-tree transformation is applied to FIR filters, the quantity of dead code increases exponentially with both the number of transformation iterations and the number of filter taps. This dead code needs to be generated before the emission of the IR code.

Without sacrificing simplicity for efficiency, the *product-elimination* transformation enables IR code for SynQ models to be generated efficiently with the directly implemented interpreter. As illustrated in Fig. 26, this transformation converts the SynQ model into the form which IR code can be generated without generating dead code introduced by projections of products. These transformed models then allow the directly implemented code generator to generate IR programs efficiently.

This transformation is similar to the *normalisation* transformation discussed in Section 6.1 and implements the following transformation rules:

$$\texttt{proj1 (prod x y)} \longmapsto \texttt{x},\ \texttt{proj2 (prod x y)} \longmapsto \texttt{y},$$
$$\texttt{app (lam x . t) (prod y z)} \longmapsto \texttt{t[(proj1 x)} \mapsto \texttt{y]},$$
$$\texttt{app (lam x . t) (prod y z)} \longmapsto \texttt{t[(proj2 x)} \mapsto \texttt{z]}.$$

In these rules, $\texttt{t[x} \mapsto \texttt{y]}$ denotes the substitution of all `x` in `t` with `y`. Aligned with the discussion in Section 6.1, the implementation is based on interpreting SynQ terms into terms of the following intermediate type, in which a product is preserved by the data constructor P:

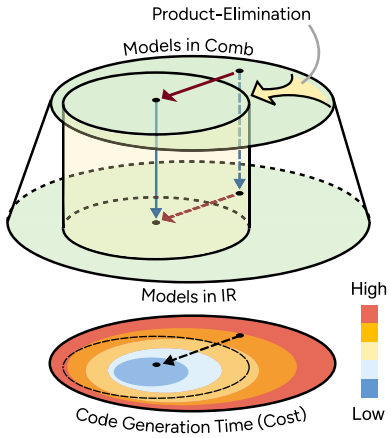


Fig. 26. The illustration of the general idea of the product-elimination transform. The solid arrows indicate the code-generation process *with product elimination*. The dashed arrow indicates the process in which the IR code is first generated and then transformed with dead-code elimination.

Table 1
A comparison of the time (in seconds) required for generating IR programs from FIR filters that are transformed by the balance-tree transformation (10/20 iterations). Platform: Intel Core i7-9750H (64 GB RAM); Dashes (–) denote timeout (2 h).

	Iters	Number of taps				
		16	32	64	128	256
Before product-elimination	10	0.079	0.077	0.086	0.171	0.866
	20	0.125	483.1	–	–	–
After product-elimination	10	0.067	0.070	0.082	0.159	0.850
	20	0.077	0.091	0.143	0.605	3.386

```
data PVal: (Type -> Type -> Type) -> Type -> Type where
  V: comb () a -> PVal comb a
  P: PVal comb a -> PVal comb b -> PVal comb (a, b)

data PView: (Type -> Type -> Type)
         -> Type -> Type -> Type where
  IsF: (PVal comb a -> PVal comb b) -> PView comb a b
  IsV: PVal comb a -> PView comb () a
```

Once the transformation is implemented, it can be applied *just as a function* in SynQ models. For instance, applying the product elimination transformation on a transformed FIR filter's reducer is conducted as follows:

```
{- same as before -}
weighted = (multKs {coefW=coefW} coefs cur)
o      = (prodElim sum) << weighted
nxt    = dropLast {aIsSig=BV {n=n}} {n=S m} cur
{- same as before -}
```

Table 1 compares the time required to generate IR programs from balanced FIR filters before and after applying the product-elimination transformation. The first two rows of the table indicate the code generation time required before the product elimination. It can be observed that the code generation time is significantly affected by the number of iterations the balance-tree transformation is applied and the number of filter taps. For instance, generating code for a 32-tap FIR filter transformed with a 20-iteration balance-tree transformation is around 6000 times slower than generating code for the same filter transformed with a 10-iteration transformation. Meanwhile, it is approximately 4000 times slower than generating code for a 16-tap filter transformed using a 20-iteration transformation. The last two rows of the table present the code generation time required after the product elimination. A significant improvement in code generation time is demonstrated, particularly for the FIR filters transformed using the 20-iteration balance-tree transformation.

8. Discussion

This section reflects on the presented framework for designing synchronous systems, covering its design choices, implications, and limitations.

SynQ is designed as a small language with few components, as shown in Sections 5.1 and 5.2, and is equipped with relatively simple interpretations in its current stage. This is evident in the two interpretations presented in Section 5.3, which operate in a context-free manner, i.e., context information in code generation is used only for generating fresh names. It follows the *minimality principle* for component-based frameworks identified in [1] and promotes accountability of design processes using the framework. A concern about the expressiveness of SynQ, however, arises.

This concern is twofold: (1) whether any desired behaviour of a system can be realised by a SynQ model; and (2) whether that behaviour can be modelled concisely and efficiently. As discussed in Section 5.1, a single-clock Lustre model [9], which can be regarded as a process network with no zero-delay feedbacks, can be transformed into a functionally equivalent SynQ model. This addresses the first part of the concern. The transformation involved (Fig. 7), however, reflects the limitation of SynQ in terms of the second part of the concern. SynQ partially mitigates this limitation through QTT's expressiveness and Idris2's high-level features, as demonstrated in Section 7.1, Section 7.2.1, and the `reduce` implementation in Section 6.2.2; however, this mitigation applies only at the modelling level. The efficiency of SynQ models remains a challenge because the limited set of components may lead to inefficient models for a given concrete platform.

At the framework level, a possible solution to this challenge is to follow the idea of the *hierarchy of languages* proposed by Turner [60]. That is, we can extend the framework with a family of EDSLs (embedded in Idris2 in the same way as SynQ), where higher-level EDSLs can be transformed into lower-level ones using vertical transformations similar to the normalisation shown in Section 6.1. This allows us to select a suitable EDSL for a given platform by choosing the transformation path from a high-level EDSL to a low-level one, thereby addressing the efficiency concern. A similar idea can be found in the platform-based design approach [61].

Another concern is the scalability of the presented framework. For modelling, SynQ, as a subset of Idris2, naturally inherits Idris2's capability of handling large and complex systems. That is, Idris2's infrastructure, e.g., the module system, parametrised namespace, export modifier, etc., can be leveraged for structuring large and complex models, as demonstrated in Section 7.1.1. The expressiveness of QTT also enables sub-models to be checked or configured by the context in which they are used, based on the typing information, as shown in Section 7.1.2.

On the other hand, the framework's capability to handle the design of large and complex systems is limited by Idris2. This is because Idris2, and its type system in particular, is not heavily optimised for performance. Consequently, type inference in the context of a large design can become a time-consuming bottleneck. The inefficiency is commonly observed in the type inference process in which evaluation of terms is involved. This is a common scenario when an intermediate step of a proof is being inferred [62]. Consequently, verifying a large design with proofs conducted manually could be time-consuming. This inefficiency also arises when SynQ terms are generated by the elaborator reflection. For instance, the elaborator script `lutGen` discussed in Section 7.1.2 must be implemented in continuation-passing style [63] to avoid the inefficiency caused by the call-by-name strategy implemented in Idris2's type checker.

To address this limitation, the domain-specific nature of the framework needs to be leveraged, allowing efficient tactics, which can be implemented with the elaborator reflection [14], to be developed for automating the verification procedure. Meanwhile, the verifiable transformations presented in Section 6 could help address scalability by decomposing a verification into an initial proof about a property, followed by proofs that each transformation preserves that property.

9. Conclusion and future work

This paper demonstrates how leveraging the expressiveness of QTT and tagless final embedding (Sections 5 and 6) addresses the challenges of building a synchronous system design framework in Idris2. The result is the well-typed EDSL SynQ that not only enables synchronous systems to be modelled and implemented, but also allows the rest of Idris2 and its QTT implementation to be leveraged for synchronous system design, verification, and transformation (Sections 4 and 7). For instance, with SynQ, the specifications of desired properties and their verification can be given together with SynQ models in Idris2. Meanwhile, transformations can be explicitly specified in Idris2 and applied to SynQ models, yielding efficient application models from directly specified ones without entering the implementation stage. This also allows an efficient design path to be chosen by combining different transformations (Section 7.3). This promotes *traceability* of a SynQ based system design framework, as advocated by Sifakis in [1]. Note that, although Idris2 is chosen for its off-the-shelf QTT support and compilation infrastructure, the approach is not Idris2-specific. This is because linear type theory is a substructural type theory and can thus be embedded in other type-theory-based proof assistants [64].

Following the discussion in Section 8, two directions of future work are identified for the presented framework. *The first* is to investigate how the expressiveness can be extended. The idea of the *hierarchy of languages* proposed by Turner [60] is promising. It, however, requires a careful selection of the language family and the transformations between them, so that the expressiveness can be extended while the accountability of design processes is maintained. This remains an open question for future exploration. *The second* is to leverage the simplicity of SynQ. Specifically, since SynQ considers only a subset of Idris2 terms, it enables us to investigate domain-specific methodologies, e.g., tactics, tailored for testing and verifying SynQ terms against the dataflow semantic model. These methodologies are expected to be more efficient than simply verifying SynQ terms as Idris2 terms. By combining these two directions, a correct-by-construction system design methodology can be expected. Meanwhile, only transformations of terms in the combinational layer of SynQ are considered in this paper. One reason is that the correctness of transformations on the sequential layer cannot be simply asserted by the propositional equality employed in this paper (Section 6.1.2). Instead, more advanced relations, such as *weak bisimulation* [65], are required, which challenge both the specification and the reasoning process. This is aligned with the first direction discussed above.

CRediT authorship contribution statement

Rui Chen: Writing – review & editing, Writing – original draft, Software, Methodology, Formal analysis, Conceptualization. **Ingo Sander:** Writing – review & editing, Supervision, Resources, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] J. Sifakis, System design automation: Challenges and limitations, Proc. IEEE 103 (2015) 2093–2103.
[2] J.Y. Girard, P. Taylor, Y. Lafont, Proofs and Types, vol. 7, Cambridge University Press, Cambridge, 1989.
[3] U. Norell, Towards a Practical Programming Language Based on Dependent Type Theory (Ph.D. thesis), 2007.
[4] L.d. Moura, S. Ullrich, The lean 4 theorem prover and programming language, in: Automated Deduction–CADE 28: 28th International Conference on Automated Deduction, July 12–15, 2021, Proceedings 28, Springer, 2021, pp. 625–635.
[5] T.C.D. Team, The coq proof assistant, 2024, http://dx.doi.org/10.5281/zenodo.14542673.
[6] E. Brady, Idris, a general-purpose dependently typed programming language: Design and implementation, J. Funct. Programming 23 (2013) 552–593.
[7] E. Brady, Idris 2: Quantitative type theory in practice, in: 35th European Conference on Object-Oriented Programming, ECOOP 2021, Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.
[8] B. Kienhuis, E.F. Deprettere, P. Van der Wolf, K. Vissers, A methodology to design programmable embedded systems: the Y-chart approach, in: International Workshop on Embedded Computer Systems, Springer, 2001, pp. 18–37.
[9] D. Pilaud, N. Halbwachs, J. Plaice, LUSTRE: A declarative language for programming synchronous systems, in: Proceedings of the 14th Annual ACM Symposium on Principles of Programming Languages, 14th POPL 1987, ACM, New York, NY, 1987, p. 188, Citeseer.
[10] A. Benveniste, P. Le Guernic, C. Jacquemot, Synchronous programming with events and relations: the SIGNAL language and its semantics, Sci. Comput. Program. 16 (1991) 103–149.
[11] C. McBride, I got plenty o'nuttin', in: A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday, 2016, pp. 207–233.
[12] R. Atkey, Syntax and semantics of quantitative type theory, in: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, 2018, pp. 56–65.
[13] J. Sifakis, Toward a system design science, in: From Programs to Systems. the Systems Perspective in Computing: ETAPS Workshop, FPS 2014, in Honor of Joseph Sifakis, Grenoble, France, April 6, 2014. Proceedings, Springer, 2014, pp. 225–234.
[14] D. Christiansen, E. Brady, Elaborator reflection: extending idris in idris, in: Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, 2016, pp. 284–297.
[15] J. Gibbons, N. Wu, Folding domain-specific languages: deep and shallow embeddings (functional pearl), in: Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming, 2014, pp. 339–347.
[16] B.C. Pierce, Types and Programming Languages, MIT Press, 2002.
[17] A. Gill, Type-safe observable sharing in haskell, in: Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell, 2009, pp. 117–128.
[18] R. Chen, I. Sander, Synchronous system design with quantitative types, Dependable Software Engineering. Theories, Tools, and Applications, Springer Nature Singapore, Singapore, 2026, pp. 13–32,
[19] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al., seL4: Formal verification of an os kernel, in: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, 2009, pp. 207–220.
[20] X. Leroy, Formal verification of a realistic compiler, Commun. ACM 52 (2009) 107–115, URL: http://xavierleroy.org/publi/compcert-CACM.pdf.
[21] T. Bourke, B. Pesin, M. Pouzet, Verified compilation of synchronous dataflow with state machines, ACM Trans. Embed. Comput. Syst. 22 (2023) 1–26.
[22] P. Caspi, G. Hamon, M. Pouzet, Synchronous functional programming: The lucid synchrone experiment, in: Real-Time Systems: Description and Verification Techniques: Theory and Tools, Hermes, 2008, pp. 28–41.
[23] S. Boulmé, G. Hamon, A clocked denotational semantics for Lucid-Synchrone in Coq, Rap. Tech. LIP6 (2001).
[24] P. Caspi, Clocks in dataflow languages, Theoret. Comput. Sci. 94 (1992) 125–140.
[25] I. Sander, A. Jantsch, System modeling and transformational design refinement in ForSyDe, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 23 (2004) 17–32.
[26] G. Ungureanu, J.E.G.D. Medeiros, T. Sundström, I. Söderquist, A. Åhlander, I. Sander, ForSyDe-atom: Taming complexity in cyber physical system design with layers, ACM Trans. Embed. Comput. Syst. (TECS) 20 (2021) 1–27.
[27] E.A. Lee, A. Sangiovanni-Vincentelli, A framework for comparing models of computation, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 17 (1998) 1217–1229.
[28] F. Bahrami, R. Jordão, I. Sander, G. Ungureanu, Automatic parallelization of embedded software via hierarchical process network transformations, in: 2024 Forum on Specification & Design Languages, FDL, IEEE, 2024, pp. 1–9.
[29] F. Bahrami, R. Jordao, I. Sander, I. Söderquist, Bridging the abstraction gap: A systematic approach to rule-based transformational design for embedded systems, ACM Trans. Embed. Comput. Syst. (2025).

[30] F. Bahrami, I. Sander, A transformation strategy for process partitioning in hierarchical concurrent process networks, J. Syst. Archit. 167 (2025) 103509.
[31] R. Jordão, F. Bahrami, R. Chen, I. Sander, A multi-view and programming language agnostic framework for model-driven engineering, in: 2022 Forum on Specification & Design Languages, FDL, IEEE, 2022, pp. 1–8.
[32] P. Bjesse, K. Claessen, M. Sheeran, S. Singh, Lava: hardware design in haskell, ACM SIGPLAN Not. 34 (1998) 174–184.
[33] A. Gill, T. Bull, G. Kimmell, E. Perrins, E. Komp, B. Werling, Introducing kansas lava, in: International Symposium on Implementation and Application of Functional Languages, Springer, 2009, pp. 18–35.
[34] S. Singh, P. James-Roxby, Lava and JBits: From HDL to bitstream in seconds, in: The 9th Annual IEEE Symposium on Field-Programmable Custom Computing Machines, FCCM'01, IEEE, 2001, pp. 91–100.
[35] A. Gill, A. Farmer, Deriving an efficient fpga implementation of a low density parity check forward error corrector, ACM SIGPLAN Not. 46 (2011) 209–220.
[36] A. Farmer, A. Gill, E. Komp, N. Sculthorpe, The HERMIT in the machine: A plugin for the interactive transformation of GHC core language programs, in: Proceedings of the 2012 Haskell Symposium, 2012, pp. 1–12.
[37] A. Megacz, Hardware design with generalized arrows, in: International Symposium on Implementation and Application of Functional Languages, Springer, 2011, pp. 164–180.
[38] T. Raudvere, I. Sander, A. Jantsch, Application and verification of local nonsemantic-preserving transformations in system design, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 27 (2008) 1091–1103.
[39] S. Abramsky, Computational interpretations of linear logic, Theoret. Comput. Sci. 111 (1993) 3–57.
[40] J.Y. Girard, Linear logic, Theoret. Comput. Sci. 50 (1987) 1–101.
[41] F. Boussinot, R. De Simone, The ESTEREL language, Proc. IEEE 79 (1991) 1293–1304.
[42] P. Gammie, Synchronous digital circuits as functional programs, ACM Comput. Surv. 46 (2013) 1–27.
[43] K. Gilles, The semantics of a simple language for parallel programming, Inf. Process. 74 (1974) 15–28.
[44] T. Bourke, L. Brun, P.É. Dagand, X. Leroy, M. Pouzet, L. Rieg, A formally verified compiler for lustre, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2017, pp. 586–601.
[45] M.P. Jones, A system of constructor classes: overloading and implicit higher-order polymorphism, in: Proceedings of the Conference on Functional Programming Languages and Computer Architecture, 1993, pp. 52–61.
[46] S.P. Jones, M. Jones, E. Meijer, Type classes: an exploration of the design space, in: Haskell Workshop, 1997, pp. 1–16.
[47] P. Hudak, J. Hughes, S. Peyton Jones, P. Wadler, A history of Haskell: being lazy with class, in: Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages, 2007, pp. 12–1–12–55.
[48] H. Barendregt, W. Dekkers, R. Statman, Lambda Calculus with Types, Cambridge University Press, 2013.
[49] N. Benton, J. Hughes, E. Moggi, Monads and effects, International Summer School on Applied Semantics, Springer, 2000, pp. 42–122.
[50] F. Pfenning, C. Elliott, Higher-order abstract syntax, ACM SIGPLAN Not. 23 (1988) 199–208.
[51] C. Lattner, V. Adve, LLVM: A compilation framework for lifelong program analysis & transformation, in: International Symposium on Code Generation and Optimization, 2004, CGO 2004, IEEE, 2004, pp. 75–86.
[52] J. Peterson, M. Jones, Implementing type classes, ACM SIGPLAN Not. 28 (1993) 227–236.
[53] J. Despeyroux, F. Pfenning, C. Schürmann, Primitive recursion for higher-order abstract syntax, in: International Conference on Typed Lambda Calculi and Applications, Springer, 1997, pp. 147–163.
[54] M. Hofmann, Semantical analysis of higher-order abstract syntax, in: Proceedings. 14th Symposium on Logic in Computer Science (Cat. No. PR00158), IEEE, 1999, pp. 204–213.
[55] G. Washburn, S. Weirich, Boxes go bananas: Encoding higher-order abstract syntax with parametric polymorphism, ACM SIGPLAN Not. 38 (2003) 249–262.
[56] S.A. Kripke, Semantical analysis of modal logic i normal modal propositional calculi, Math. Log. Q. 9 (1963) 67–96.
[57] D. Siafarikas, J.L. Volakis, Toward direct RF sampling: Implications for digital communications, IEEE Microw. Mag. 21 (2020) 43–52.
[58] E. Hogenauer, An economical class of digital filters for decimation and interpolation, IEEE Trans. Acoust. Speech Signal Process. 29 (1981) 155–162.
[59] P. Baudin, J.C. Filliâtre, C. Marché, B. Monate, Y. Moy, V. Prevosto, ACSL: ANSI/ISO C specification, 2021, URL https://frama-c.com/html/acsl.html.
[60] D.A. Turner, Total functional programming, J. Univers. Comput. Sci. 10 (2004) 751–768.
[61] A. Sangiovanni-Vincentelli, G. Martin, Platform-based design and software design methodology for embedded systems, IEEE Des. Test Comput. 18 (2001) 23–33.
[62] A. Abel, T. Coquand, P. Dybjer, Normalization by evaluation for Martin–Löf type theory with typed equality judgements, in: 22nd Annual IEEE Symposium on Logic in Computer Science, LICS 2007, IEEE, 2007, pp. 3–12.
[63] J.C. Reynolds, The discoveries of continuations, Lisp Symb. Comput. 6 (1993) 233–247.
[64] J. Polakow, Embedding a full linear lambda calculus in haskell, ACM SIGPLAN Not. 50 (2015) 177–188.
[65] D. Pous, New up-to techniques for weak bisimulation, Theoret. Comput. Sci. 380 (2007) 164–180.



Rui Chen received the B.Eng degree in Electronic Information Engineering from Dalian University of Technology, Dalian, China. His currently pursuing a Ph.D. at KTH Royal Institute of Technology in Sweden. His research focuses on design methodologies for embedded systems, with particular interest in system modelling and type theory/functional programming based framework.



Ingo Sander received the M.Sc. degree in Electrical Engineering from the Technical University of Braunschweig, Germany, in 1990 and the Ph.D. degree and docent degree from KTH Royal Institute of Technology, Sweden, in 2003 and 2009, respectively. Between 1991 and 1993 he worked as system design engineer at Ericsson, Sweden.

In 1993 he joined KTH, where he since 2018 holds a position as professor in Electronic System Design. His main research interests are located in the area of design methodologies for embedded systems including system modelling, design space exploration and system synthesis.